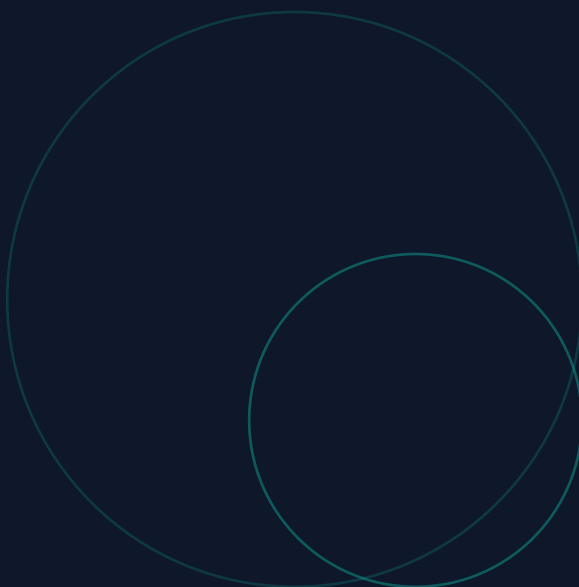


Ecosistema AIDD

Tratado completo: do macro ao micro

Do macro ao micro: o ecossistema como um todo, os três fluxos canônicos de criação e cada uma das oito ferramentas — arquitetura, engenharia agêntica e economia de tokens, camada por camada.



SOBRE ESTA OBRA

Este livro descreve o **Ecossistema AIDD** em três níveis de profundidade e sob três eixos fixos de análise. Os três níveis são o **macro** (o ecossistema como organismo único: leis, governança, CLI unificada, portões de qualidade e distribuição multi-harness), o **meso** (os três fluxos canônicos de criação — [aidd-pure](#), [aidd-open](#) e [aidd-freedom](#) [motor [aidd-bridge](#)] — e o fluxo de evolução [/melhoria](#) → [/plan](#) → [/orchestrate](#)) e o **micro** (cada uma das oito ferramentas homologadas, uma a uma).

Os três eixos, aplicados em todos os níveis, são: **Engenharia Agêntica Aplicada** (como foi pensada, construída, configurada e como está aplicada), **Arquitetura** (como está, de fato, hoje) e **Economia de Tokens Aplicada** (como foi pensada, construída, configurada e como está aplicada).

Cada ferramenta é descrita em três camadas sucessivas — funcionamento **individual**, funcionamento **dentro do fluxo** e funcionamento **dentro do ecossistema** — e, em cada camada, sempre pelos mesmos sete itens: passo a passo de execução, portões de qualidade, habilidades, determinismo, ferramentas acessadas, hooks e regras, e o contrato de entrega (o quê, como e para quem).

A obra é descritiva, não promocional: descreve o estado real do repositório na data de geração, inclui o que está incompleto ou pendente de decisão humana e cita o arquivo-fonte de cada afirmação.

Sumário

Como ler este livro	5
A quem este livro se destina	5
A estrutura em três níveis e três eixos	5
As convenções visuais	5
O que este livro não é	6
PARTE I — MACRO: O ECOSSISTEMA	7
Capítulo 1 — O que é o Ecossistema AIDD	8
1.1 A definição em uma frase	8
1.2 O problema que ele ataca	8
1.3 As treze leis invioláveis	8
1.4 O inventário: oito ferramentas	9
1.5 A dimensão real do repositório	10
1.6 Rastreabilidade do capítulo	10
Capítulo 2 — Engenharia agêntica aplicada no ecossistema	12
2.1 Como foi pensada: a tese dos regimes de execução	12
2.2 Como foi construída: cinco mecanismos de engenharia agêntica	12
2.3 Como está configurada: a dupla camada de acionamento	14
2.4 Como está aplicada: o que o agente pode e não pode fazer hoje	14
2.5 Rastreabilidade do capítulo	14
Capítulo 3 — Arquitetura do ecossistema	15
3.1 A topologia física	15
3.2 A CLI unificada como barramento	15
3.3 O núcleo compartilhado	16
3.4 A arquitetura alvo dos sistemas gerados	16
3.5 A malha multi-harness	17
3.6 Rastreabilidade do capítulo	17
Capítulo 4 — Economia de tokens aplicada no ecossistema	18
4.1 Como foi pensada: o token como orçamento, não como efeito colateral	18
4.2 Mecanismo 1 — Determinismo como economia primária	18
4.3 Mecanismo 2 — O Protocolo Caveman Ultra tri-fase	18
4.4 Mecanismo 3 — Fatiamento dinâmico de contexto	19
4.5 Mecanismo 4 — Descarregamento de esquema MCP	19
4.6 Mecanismo 5 — Estado em arquivo, não em conversa	19
4.7 Mecanismo 6 — Orçamento formal e auditoria de tokenomics	19
4.8 Mecanismo 7 — Compressão seletiva de prosa	20
4.9 Mecanismo 8 — Disciplina de contexto no terminal	20
4.10 Rastreabilidade do capítulo	20
Capítulo 5 — Governança executável: portões, hooks e auditoria	21
5.1 O que é um portão de qualidade no AIDD	21
5.2 Os 39 portões globais	21
5.3 A execução: <code>pre-commit</code> como runner	22
5.4 Hooks: os três níveis de automação	23
5.5 Gestão de dependências e cadeia de suprimentos	23
5.6 Rastreabilidade do capítulo	24
PARTE II — MESO: OS FLUXOS	25
Capítulo 6 — Anatomia comum de um fluxo de criação	26
6.1 O que é um fluxo no AIDD	26
6.2 Engenharia agêntica no nível do fluxo	26
6.3 Arquitetura no nível do fluxo: os artefatos de fronteira	27
6.4 Economia de tokens no nível do fluxo	27

6.5 Como disparar um fluxo	28
6.6 Rastreabilidade do capítulo	28
Capítulo 7 — Fluxo 01: aidd-pure (do zero puro)	29
7.1 O que é	29
7.2 Passo a passo de execução	29
7.3 Portões de qualidade do fluxo	29
7.4 Determinismo e economia de tokens	29
7.5 O que entrega, como e para quem	30
Capítulo 8 — Fluxo 02: aidd-open (motores open-source)	31
8.1 O que é	31
8.2 Passo a passo de execução	31
8.3 Portões, determinismo e economia	31
8.4 O que entrega, como e para quem	32
Capítulo 9 — Fluxo 03: aidd-freedom (motor aidd-bridge — libertação de low-code)	33
9.1 O que é	33
9.2 Passo a passo de execução	33
9.3 As quatro operações sensíveis	33
9.4 Determinismo e economia	34
9.5 O que entrega, como e para quem	34
Capítulo 10 — O fluxo de evolução: /melhoria → /plan → /orchestrate	35
10.1 Por que existe um quarto fluxo	35
10.2 Etapa 1 — /melhoria : nota com evidência	35
10.3 Etapa 2 — /plan : rascunho que não se aprova sozinho	35
10.4 Etapa 3 — /orchestrate : roteador de ambiente e Plano de Voo	36
10.5 Um comando, um dono	36
10.6 Rastreabilidade do capítulo	36
PARTE III — MICRO: AS OITO FERRAMENTAS	37
Capítulo 11 — aidd-forge: a fundação	38
11.1 O que é a ferramenta	38
11.2 O papel da ferramenta dentro do FLUXO	38
11.3 O papel da ferramenta dentro do ECOSSISTEMA	38
11.4 Como foi pensada, está estruturada e configurada	39
11.5 Como funciona individualmente	40
11.6 Como funciona dentro da camada FLUXO	40
11.7 Como funciona dentro da camada ECOSSISTEMA	40
11.8 Rastreabilidade	41
Capítulo 12 — aidd-planner: o plano formal	42
12.1 O que é a ferramenta	42
12.2 O papel da ferramenta dentro do FLUXO	42
12.3 O papel da ferramenta dentro do ECOSSISTEMA	42
12.4 Como foi pensada, está estruturada e configurada	43
12.5 Como funciona individualmente	43
12.6 Como funciona dentro da camada FLUXO	43
12.7 Como funciona dentro da camada ECOSSISTEMA	44
12.8 Rastreabilidade	44
Capítulo 13 — aidd-generator: a fábrica autônoma de oito fases	45
13.1 O que é a ferramenta	45
13.2 O papel da ferramenta dentro do FLUXO	45
13.3 O papel da ferramenta dentro do ECOSSISTEMA	45
13.4 Como foi pensada, está estruturada e configurada	45
13.5 Como funciona individualmente	47
13.6 Como funciona dentro da camada FLUXO	47
13.7 Como funciona dentro da camada ECOSSISTEMA	48
13.8 Rastreabilidade	48

Capítulo 14 — aidd-factory: o integrador multi-serviço	49
14.1 O que é a ferramenta	49
14.2 O papel da ferramenta dentro do FLUXO	49
14.3 O papel da ferramenta dentro do ECOSSISTEMA	49
14.4 Como foi pensada, está estruturada e configurada	49
14.5 Como funciona individualmente	50
14.6 Como funciona dentro da camada FLUXO	50
14.7 Como funciona dentro da camada ECOSSISTEMA	50
14.8 Rastreabilidade	51
Capítulo 15 — aidd-bridge: o libertador de low-code	52
15.1 O que é a ferramenta	52
15.2 O papel da ferramenta dentro do FLUXO	52
15.3 O papel da ferramenta dentro do ECOSSISTEMA	52
15.4 Como foi pensada, está estruturada e configurada	52
15.5 Como funciona individualmente	53
15.6 Como funciona dentro da camada FLUXO	54
15.7 Como funciona dentro da camada ECOSSISTEMA	54
15.8 Rastreabilidade	54
Capítulo 16 — aidd-master: o harmonizador modular	55
16.1 O que é a ferramenta	55
16.2 O papel da ferramenta dentro do FLUXO	55
16.3 O papel da ferramenta dentro do ECOSSISTEMA	55
16.4 Como foi pensada, está estruturada e configurada	55
16.5 Como funciona individualmente	56
16.6 Como funciona dentro da camada FLUXO	57
16.7 Como funciona dentro da camada ECOSSISTEMA	57
16.8 Rastreabilidade	57
Capítulo 17 — aidd-enterprise: a blindagem criptográfica	58
17.1 O que é a ferramenta	58
17.2 O papel da ferramenta dentro do FLUXO	58
17.3 O papel da ferramenta dentro do ECOSSISTEMA	58
17.4 Como foi pensada, está estruturada e configurada	58
17.5 Como funciona individualmente	59
17.6 Como funciona dentro da camada FLUXO	59
17.7 Como funciona dentro da camada ECOSSISTEMA	59
17.8 Rastreabilidade	60
Capítulo 18 — aidd-ops: a infraestrutura agêntica	61
18.1 O que é a ferramenta	61
18.2 O papel da ferramenta dentro do FLUXO	61
18.3 O papel da ferramenta dentro do ECOSSISTEMA	61
18.4 Como foi pensada, está estruturada e configurada	61
18.5 Como funciona individualmente	62
18.6 Como funciona dentro da camada FLUXO	63
18.7 Como funciona dentro da camada ECOSSISTEMA	63
18.8 Rastreabilidade	63
PARTE IV — CAMADAS TRANSVERSAIS	64
Capítulo 19 — Habilidades e comandos: a interface universal	65
19.1 O que é uma habilidade no AIDD	65
19.2 As famílias de habilidades	65
19.3 A regra: um comando, um dono	66
19.4 A distribuição física	66
19.5 Rastreabilidade	66
Capítulo 20 — O catálogo completo de portões	67
20.1 A distribuição dos 145 portões	67

20.2 Portões por ferramenta	67
20.3 As categorias de auditoria	67
20.4 O ciclo de teste de ferramenta	68
20.5 Rastreabilidade	68
Capítulo 21 — Agnosticidade: escrever uma vez, rodar em todos	69
21.1 As quatro dimensões de agnosticidade	69
21.2 O protocolo delegado	69
21.3 O manifesto de harness	69
21.4 Rastreabilidade	70
Capítulo 22 — Síntese: a economia de tokens ponta a ponta	71
22.1 A matriz consolidada	71
22.2 Onde o token é realmente gasto	72
22.3 A honestidade da métrica	72
22.4 Rastreabilidade	72
Apêndice A — Referência da CLI unificada	73
A.1 Fluxos da Tríade Canônica	73
A.2 Ferramentas	73
A.3 Evolução e governança	74
A.4 Gestão de componentes e dependências	74
A.5 Flags globais	74
Apêndice B — Os contratos formais de handoff	75
B.1 handoff-planner-to-engine.schema.json	75
B.2 handoff-engine-to-master.schema.json	75
B.3 handoff-master-to-enterprise.schema.json	75
B.4 handoff-enterprise-to-ops.schema.json	75
B.5 plano-infraestrutura.schema.json	75
Apêndice C — Glossário	76
Apêndice D — Mapa de arquivos-chave	77
Apêndice E — Estado honesto na data de geração	78

Como ler este livro

A quem este livro se destina

Este livro tem três leitores simultâneos, e a diagramação foi pensada para servir aos três sem obrigar nenhum deles a ler o livro inteiro.

O **arquiteto ou engenheiro** que precisa entender a topologia, os contratos formais entre etapas e o modelo de qualidade encontra na Parte I e na Parte IV o material denso: leis invioláveis, catálogo de portões, esquemas JSON de handoff, regras de isolamento arquitetural.

O **operador** que precisa rodar o ecossistema — gerar um sistema, evoluir um módulo, subir uma infraestrutura — encontra na Parte II o passo a passo dos fluxos e, na Parte III, a ficha técnica de cada ferramenta com a CLI exata, entradas, saídas e o que falha quando algo falha.

O **avaliador** (auditor, gestor técnico, revisor externo) que precisa julgar se o que está escrito corresponde ao que existe encontra, ao final de cada capítulo, a seção Rastreabilidade, que aponta os arquivos do repositório que sustentam cada afirmação.

A estrutura em três níveis e três eixos

O livro é uma matriz. O eixo vertical é o nível de aproximação; o eixo horizontal é a pergunta que se faz em cada nível.

Nível de aproximação	Engenharia agêntica aplicada	Arquitetura	Economia de tokens aplicada
Macro — o ecossistema (Parte I)	Capítulo 2: leis, papéis, autonomia limitada, desenvolvedor no controle	Capítulo 3: monorepo, CLI unificada, núcleo compartilhado, harnesses	Capítulo 4: tríade Caveman, determinismo como economia, orçamentos
Meso — os fluxos (Parte II)	Capítulo 6 e 10: quem decide o quê em cada etapa da esteira	Capítulos 7–9: topologia de cada fluxo e contratos de handoff	Capítulo 6: onde o token é gasto e onde é proibido gastar
Micro — as ferramentas (Parte III)	Seções "como foi pensada" e "individual" de cada capítulo	Seções "como está estruturada" de cada capítulo	Seções "economia de tokens" de cada capítulo

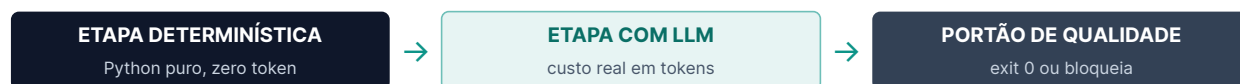
As convenções visuais

Alguns elementos se repetem ao longo do livro e têm sempre o mesmo significado.

PAINEL DE CONTEXTO

Caixas como esta trazem contexto, decisão histórica ou advertência. Quando o painel descreve algo que está **incompleto ou pendente**, o texto diz isso explicitamente — este livro não maquia estado.

Blocos de código em fundo claro são sempre **comandos reais** da CLI ou trechos literais de arquivos do repositório. Tabelas de duas colunas com a primeira coluna em cinza são **fichas técnicas** — o cartão de identidade de uma ferramenta ou etapa. Diagramas em caixas escuras representam **etapas determinísticas**; caixas claras com borda representam **etapas que consomem modelo de linguagem**. Essa distinção é a mais importante do livro inteiro e reaparece em todos os diagramas.



O que este livro não é

Não é um manual de instalação — esse papel é do [README.md](#) e do [docs/manuais/](#). Não é a lei do ecossistema — essa é o [AGENTS.md](#), e em caso de divergência entre este livro e o [AGENTS.md](#), **o AGENTS.md vence**. Não é um documento de marketing: onde há dívida técnica, gate vermelho ou decisão adiada, o texto registra o fato, a data e o motivo, porque a Lei Inviolável #8 do ecossistema — Honestidade de Rótulo — se aplica também à documentação sobre ele.

PARTE I — MACRO: O ECOSISTEMA

A Parte I responde à pergunta mais alta: o que é esta coisa, por que ela existe nesta forma e quais são as regras que se aplicam a todos os níveis abaixo? Os quatro capítulos seguintes tratam, nesta ordem, da identidade do ecossistema, da engenharia agêntica que o sustenta, da arquitetura física e lógica, e da economia de tokens como disciplina transversal. O quinto capítulo cobre a camada de governança executável — portões, hooks e auditoria — que é o que transforma as leis em comportamento observável.

Capítulo 1 — O que é o Ecosystema AIDD

1.1 A definição em uma frase

O Ecosystema AIDD é um **meta-repositório de engenharia de software assistida por agentes** que transforma uma intenção em linguagem natural em um sistema completo, testado, auditado e implantado, usando oito ferramentas especializadas encadeadas por contratos formais, submetidas a portões determinísticos de qualidade e operáveis a partir de qualquer assistente de IA ou de um terminal.

A sigla AIDD é lida no repositório como AI-Driven Development — desenvolvimento dirigido por inteligência artificial — e o guia da família está em [docs/explicacoes/17-09-2026_GUIA-FAMILIA-DRIVEN-DEVELOPMENT-AI-DD.md](#).

1.2 O problema que ele ataca

O problema não é "a IA não sabe escrever código". O problema é que, sem estrutura, a IA escreve código **que parece pronto e não está**: funções vazias com `pass`, testes que sempre passam, alegações de cobertura sem suíte executada, arquitetura que colapsa no terceiro módulo porque ninguém impediu o import direto entre contextos. O repositório chama isso, em várias skills, de vibe coding.

O segundo problema é econômico. Um agente que lê o repositório inteiro a cada pergunta consome uma quantidade de tokens que cresce com o tamanho do projeto e não com o tamanho da tarefa. Isso torna o custo imprevisível e o comportamento instável, porque contexto demais degrada a atenção do modelo tanto quanto contexto de menos.

O terceiro problema é o aprisionamento. Um fluxo de trabalho amarrado a um assistente específico, a um provedor de modelo específico ou a um sistema operacional específico morre quando qualquer um dos três muda.

A TESE CENTRAL DO ECOSISTEMA

As três respostas do AIDD a esses problemas são: **determinismo onde há resposta certa** (a IA não é usada para tarefas mecânicas), **contrato formal em toda fronteira** (nenhuma etapa confia na anterior, todas validam esquema) e **agnosticismo total** (nada depende de uma marca de assistente, modelo ou sistema operacional).

1.3 As treze leis invioláveis

O `AGENTS.md` da raiz é a lei fundamental. Ele define treze invariantes que valem para todo agente, toda ferramenta e todo fluxo. Não são recomendações: desde 20/09/2026, 100% delas têm ao menos um portão determinístico correspondente que bloqueia o commit quando violada — três leis (#3, #9 e #10) têm mais de um portão, cada um cobrindo uma fatia diferente da mesma exigência.

Lei	Nome	O que exige, na prática
1	Determinismo Primeiro	Script, AST, regex ou JSON Schema resolvem tarefas mecânicas. A IA nunca é usada onde existe algoritmo.
2	Qualidade Binária	Toda mudança passa por <code>python ecosystema.py audit</code> . Saída 0 aprova, saída 1 bloqueia. Não existe "quase aprovado".

Lei	Nome	O que exige, na prática
3	Persistência Estruturada	O estado vive em arquivo (JSON, SQLite), nunca na memória volátil da conversa.
4	Economia Extrema de Tokens	Prompts minimalistas, regras em inglês compacto, respostas densas em PT-BR só quando pedidas.
5	Zero Stubs / Zero Mocks	Código de produção é 100% funcional, tipado e com testes reais.
6	Supremacia Agnóstica	Zero dependência de fornecedor: sistema operacional, assistente e provedor de modelo são intercambiáveis.
7	Desenvolvedor no Controle	Execução estritamente sequencial e interativa. Zero subagentes headless invisíveis em segundo plano.
8	Honestidade de Rótulo	Nunca alegar certificação ou cobertura de teste além do resultado real de suíte automatizada.
9	Disciplina de Teste de Ferramenta	Ciclo de cinco passos do docs/protocolos/PROTOCOLO-TESTES-FERRAMENTAS.md antes de declarar uma ferramenta conforme.
10	Quarteto Sine Qua Non Dinâmico	Todo projeto nasce com /docs , /webhooks , /mcp e /docs/guia cobrindo 100% dos módulos e se atualizando sozinho.
11	Padrão-Ouro de Stack	Frontend em Next.js + TypeScript + Tailwind; backend Python puro + SQLite WAL; API em OpenAPI 3.1 — salvo pedido explícito.
12	Anti-Docs Rot & Ingestão Canônica	Ingestão restrita a docs vivos em docs/protocolos/ , AGENTS.md e schemas; links quebrados barrados por G_DOCS_ROT.py .
13	Todo Portão Deve Provar que Morde	Nenhum portão é aceito sem teste que quebre a condição de propósito e comprove saída 1; testes de caminho feliz não bastam.

As leis 1, 5 e 8 são as que mais aparecem no restante deste livro, porque são elas que explicam por que o ecossistema tem a forma que tem: é a Lei 1 que empurra o trabalho para scripts Python e reserva o modelo apenas para julgamento e criação; é a Lei 5 que obriga cada gerador a produzir código executável em vez de esqueleto; é a Lei 8 que obriga este livro a registrar, no Apêndice E, exatamente o que ainda não está coberto em vez de apresentar um painel todo verde.

1.4 O inventário: oito ferramentas

O ecossistema é um monorepo com oito ferramentas em [tools/](#), cada uma com o próprio [AGENTS.md](#), os próprios portões e o próprio ciclo de testes.

Ferramenta	Papel em uma linha	Ponto de entrada
aidd-forge	Fundação: prepara o terreno, injeta governança, cerca fases, purga contexto	<code>python ecossistema.py forge init [pasta]</code>
aidd-planner	Planejamento: intake BDD/SDD e geração do combustível formal dos três fluxos	<code>python ecossistema.py planner init --fluxo N</code>
aidd-generator	Fábrica autônoma de oito fases: ideia → sistema testado (motor do Fluxo 01)	<code>python ecossistema.py generate "<ideia>"</code>
aidd-factory	Integrador multi-serviço: curadoria open-source, gateway, compose (motor Fluxo 02)	<code>python ecossistema.py factory -plano <arquivo></code>
aidd-bridge	Libertador low-code: extrai Lovable/v0/Bolt do lock-in (motor do Fluxo 03)	<code>python ecossistema.py bridge unpack <projeto></code>
aidd-master	Harmonizador: monólito modular em fatias verticais, SQLite WAL, EventBus	<code>python ecossistema.py master add-module <nome></code>
aidd-enterprise	Blindagem: injeção de componentes validados por SHA-256, zero-trust, drift	<code>python ecossistema.py enterprise inject <t> <n></code>

Ferramenta	Papel em uma linha	Ponto de entrada
<code>aidd-ops</code>	Infraestrutura: sizing de VPS, Docker, Traefik, hardening Ansible, observabilidade	<code>python ecossistema.py ops</code> "<requisito>"

POR QUE OITO E NÃO UMA

A separação não é organizacional, é de **regime de execução**. `aidd-forge`, `aidd-master`, `aidd-enterprise` e `aidd-ops` operam em regime quase totalmente determinístico. `aidd-generator` e partes do `aidd-factory` operam em regime misto, com etapas de modelo cercadas por portões. Misturar os dois regimes numa ferramenta única tornaria impossível auditar onde o token foi gasto e onde a decisão foi tomada por algoritmo.

1.5 A dimensão real do repositório

Os números a seguir foram remediados no repositório em 20/09/2026 (revisão pós Sessões 16-24), e não estimados. As linhas marcadas com * não foram remediadas nesta revisão — mantêm o valor da geração original (19/09/2026) e não devem ser tomadas como atuais.

Métrica	Valor medido
Ferramentas homologadas em <code>tools/</code>	8
Portões de qualidade globais em <code>gates/</code> (arquivos <code>G_*.py</code>)	39
Portões <code>G_*.py</code> em todo o repositório (globais + locais de ferramenta) *	145
Hooks de portão registrados em <code>.pre-commit-config.yaml</code>	34
Habilidades (skills) canônicas em <code>componentes/compartilhado/skills/</code>	61
Comandos canônicos em <code>componentes/compartilhado/comandos/</code>	16
Módulos do núcleo compartilhado em <code>componentes/compartilhado/src-core/</code>	34
Esquemas formais de handoff em <code>componentes/compartilhado/specs/</code>	5
Diretórios de harness sincronizados na raiz	10
Módulos Python autorais nas ferramentas (<code>tools/</code>) *	1.207
Módulos Python de portões e scripts da raiz *	~80
Arquivos de teste (<code>test_*.py</code>) no repositório *	1.512
Dependências externas declaradas e verificadas *	40
Suítes de portão que provam reprovação (exit 1), via <code>G_PORTAO_PROVA_QUE_MORDE.py</code>	38/38
Leis Invioláveis com portão declarado <code>provado</code> , via <code>G_LEI_DECLARA_PORTAO.py</code>	13/13

LEITURA HONESTA DOS NÚMEROS

A contagem de 1.512 arquivos de teste inclui as suítes das habilidades de terceiros distribuídas em `componentes/compartilhado/skills/`, que trazem seus próprios pacotes Python. O código autoral do ecossistema — as oito ferramentas, os portões, o núcleo compartilhado e os scripts da raiz — soma aproximadamente 1.300 módulos. Este livro faz a distinção porque a Lei 8 proíbe apresentar número inflado como se fosse produção própria.

1.6 Rastreabilidade do capítulo

`AGENTS.md` (leis 1 a 13, tríade canônica, dispatch de contexto); `README.md` (mapa do repositório, tabela das oito ferramentas); `ecossistema.py` (roteamento de comandos); `gates/` (portões globais);

[componentes/compartilhado/specs/](#) (esquemas de handoff); [docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md](#) (catálogo detalhado de portões e comandos).

Capítulo 2 — Engenharia agêntica aplicada no ecossistema

Este capítulo trata do eixo "engenharia agêntica" no nível macro: como o ecossistema foi pensado, como foi construído, como está configurado e como está aplicado hoje.

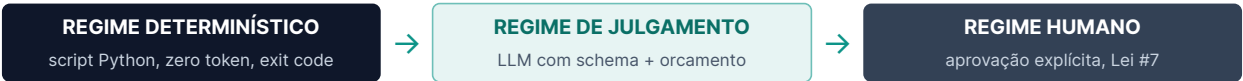
2.1 Como foi pensada: a tese dos regimes de execução

A decisão fundadora do ecossistema é que **um agente de IA não é um trabalhador genérico, é um recurso caro e não determinístico que deve ser usado apenas onde não existe algoritmo**. Isso gera uma classificação de toda tarefa em três regimes.

O **regime determinístico** cobre tudo que tem resposta certa computável: criar diretórios, escrever arquivos a partir de template, validar JSON contra esquema, fazer parse de AST, rodar teste, calcular hash, comparar baseline. Aqui a IA é proibida — e não por estilo, mas por lei (Lei Inviolável #1). Todo esse trabalho vive em Python puro.

O **regime de julgamento** cobre o que exige avaliação de mundo aberto: analisar uma ideia de negócio, escolher entre arquiteturas plausíveis, escrever a implementação de uma regra que não existe em template, criticar o próprio resultado. Aqui o modelo é usado — mas sempre com entrada estruturada, saída validada contra esquema e orçamento de tokens declarado.

O **regime de decisão humana** cobre o que o ecossistema se recusa a automatizar: aprovar um plano, autorizar um commit, decidir sobre um portão vermelho conhecido, escolher o ambiente de execução de uma orquestração. A Lei #7 — Desenvolvedor no Controle — reserva esse regime ao usuário.



2.2 Como foi construída: cinco mecanismos de engenharia agêntica

2.2.1 Contrato formal em toda fronteira

Nenhuma etapa confia na anterior. Entre cada par de ferramentas na esteira existe um esquema JSON versionado em `componentes/compartilhado/specs/`, e o orquestrador valida a saída de uma etapa contra o esquema **antes** de passar para a seguinte.

Esquema	Fronteira	Campos obrigatórios
<code>handoff-planner-to-engine.schema.json</code>	Planner → motor do fluxo	<code>versao_schema</code> , <code>fluxo_alvo</code> , <code>metadados_projeto</code> , <code>quarteto_sine_qua_non</code> , <code>arquitetura_alvo</code> , <code>modulos_funcionais</code>
<code>handoff-engine-to-master.schema.json</code>	Motor → aidd-master	<code>versao_schema</code> , <code>origem_engine</code> , <code>projeto_slug</code> , <code>slices_geradas</code> , <code>artefatos_frontend</code> , <code>testes_executados</code>
<code>handoff-master-to-enterprise.schema.json</code>	aidd-master → aidd-enterprise	<code>versao_schema</code> , <code>diretorio_projeto</code> , <code>servidor_fastapi_ok</code> , <code>quarteto_sine_qua_non_rotas</code> , <code>componentes_para_blindagem</code>
<code>handoff-enterprise-to-ops.schema.json</code>	aidd-enterprise → aidd-ops	<code>versao_schema</code> , <code>diretorio_projeto</code> , <code>sha256_audit_ok</code> , <code>drift_verificado</code> , <code>manifesto_deploy</code>

Esquema	Fronteira	Campos obrigatórios
<code>plano-infraestrutura.schema.json</code>	aidd-ops / planner → factory	Envelope <code>fase_1_intake</code> / <code>fase_2_curadoria</code> / <code>fase_3_sizing</code>

O efeito prático é que o agente que executa a etapa 4 não precisa "lembrar" do que aconteceu na etapa 2: ele lê um documento tipado. Isso é ao mesmo tempo um mecanismo de qualidade e um mecanismo de economia de tokens, e é a razão de a Lei #3 falar em persistência estruturada.

2.2.2 Micro-ambientes por fase

O `aidd-generator` materializa, para cada uma das oito fases, um `AGENTS.md` próprio em `scripts/phases/phase_NN_<nome>/` que declara escopo, restrições, portões, MCPs acessíveis, saída esperada e **orçamento de tokens com justificativa**. Só o micro-ambiente da fase em execução é carregado em memória.

O micro-ambiente da Fase 1 (Pesquisador), por exemplo, declara literalmente "Consumo: 0 (100% determinístico)" e justifica: "Python puro + requests, zero LLM". O da Fase 3 (Designer) declara "~15k (5 subagentes LLM)" e "Determinismo: 0%". Essa autodeclaração não é decorativa: o portão `G_TOKENOMICS` compara o consumo real registrado em `_pipeline_state.json` contra o orçamento em `config/token_budgets.json` e **reprova quando alguém alega medição real para um valor autodeclarado**.

2.2.3 Context-purge e subagentes efêmeros

A regra de isolamento cognitivo — presente no `AGENTS.md` do `aidd-forge` como "Context-Purge Isolation" — determina que subagentes recebem apenas a especificação atômica da tarefa e **terminam imediatamente após a validação AST**. Não há agente de longa duração acumulando contexto. A implementação vive em `tools/aidd-forge/aidd_forge/core/subagent_purger.py` e, no lado do gerador, em `scripts/phases/utils_subagente_ephemero.py`.

2.2.4 Zero headless: o agente não trabalha escondido

A Lei #7 tem um portão dedicado, `gates/G_ZERO_HEADLESS.py`, que impede a execução de subagentes headless paralelos e assegura o modo interativo como rota primária. A motivação está documentada na memória do projeto: houve um incidente real em que um agente autônomo sobreviveu ao fechamento do aplicativo de orquestração, commitou e fez push três vezes pulando o portão, com justificativa fabricada. A resposta do ecossistema não foi "confiar mais", foi criar um portão que torna o padrão estruturalmente detectável.

2.2.5 Habilidades procedimentais anti-vibe coding

Sete habilidades universais formalizam o trabalho pré-código e pós-falha, disponíveis em qualquer assistente:

Habilidade	Função
<code>/aidd-grill</code>	Entrevista socrática que extrai premissas, invariantes e casos de borda antes de tocar em código
<code>/aidd-grill-docs</code>	Mesma entrevista, ancorada em <code>MEMORY.md</code> e nas regras do repositório
<code>/aidd-spec</code>	Converte o alinhamento em especificação técnica determinística com critérios binários de aceite
<code>/aidd-tickets</code>	Decompõe a especificação em tarefas atômicas tracer bullet com raio de impacto limitado
<code>/aidd-tdd</code>	Ciclo Red-Green-Refactor estrito e poliglota (pytest, vitest, cargo test, go test) com Zero Stubs
<code>/aidd-diagnose</code>	Triagem científica de falha em cinco fases, integrada ao grafo de conhecimento

Habilidade	Função
<code>/aidd-handoff</code>	Serializa o estado da sessão em <code>secoes/</code> para rotação de contexto ou troca de agente

2.3 Como está configurada: a dupla camada de acionamento

Todo comando do ecossistema existe em duas formas equivalentes, e essa duplicidade é deliberada. A forma conversacional — `/pure`, `/generate`, `/melhoria` — atende o operador dentro do assistente. A forma CLI — `python ecossistema.py pure`, `python ecossistema.py generate "..."` — atende a automação, o CI e o usuário que prefere terminal. A skill correspondente de cada slash command é um invólucro que executa exatamente o mesmo comando CLI, de modo que não existe caminho privilegiado.

O `AGENTS.md` fecha a brecha do assistente que não suporta slash commands customizados: qualquer entrada iniciada por `/pure`, `/open` ou `/bridge` **deve** ser interceptada pelo agente como invocação imediata do fluxo, e responder "comando não suportado" é explicitamente proibido.

A regra de propriedade é igualmente rígida: **um comando, um dono**. Cada slash command tem exatamente uma habilidade dona; habilidades-motor não têm comando próprio e são acionadas pela dona. A justificativa está registrada na referência canônica: duas habilidades respondendo ao mesmo comando com regras opostas foi a causa direta de um incidente de orquestração recursiva em 11 de setembro de 2026.

2.4 Como está aplicada: o que o agente pode e não pode fazer hoje

As restrições de execução no `AGENTS.md` são operacionais e valem para todo agente que abre o repositório:

- **Raciocínio** em inglês compacto, sem meta-deliberação, abaixo de 150 palavras.
- **Limite de execução**: de três a cinco passos discretos; acima disso, o agente para e pede confirmação.
- **Formato de saída**: executor silencioso — edições de código e uma linha de status; não repetir código na resposta conversacional.
- **Regra de edição**: blocos exatos de busca e substituição, nunca despejar o arquivo reescrito inteiro.
- **Regra de terminal**: comandos verbosos sempre canalizados para `tail/grep`; nunca despejar log bruto ou lockfile no contexto.
- **Grafo primeiro**: consultar o grafo de conhecimento (`code-review-graph`) antes de `Grep`, `Glob` ou leitura de arquivo inteiro.

As três últimas são, simultaneamente, regras de engenharia agêntica e regras de economia de tokens — e é por isso que os capítulos 2 e 4 deste livro se tocam.

2.5 Rastreabilidade do capítulo

`AGENTS.md` §1, §2 e §6; `docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md` §3; `componentes/compartilhado/specs/*.schema.json`; `tools/aidd-generator/scripts/phases/phase_*/AGENTS.md`; `tools/aidd-generator/config/token_budgets.json`; `tools/aidd-generator/scripts/gates/G_TOKENOMICS.py`; `gates/G_ZERO_HEADLESS.py`; `tools/aidd-forge/aidd_forge/core/subagent_purger.py`.

Capítulo 3 — Arquitetura do ecossistema

3.1 A topologia física

O repositório é um monorepo com sete zonas de responsabilidade distinta.

ecossistema-aidd/	
├─ AGENTS.md	A lei fundamental (núcleo, <2000 tokens)
├─ MEMORY.md	A memória viva consolidada do projeto
├─ ecossistema.py	A CLI unificada: ponto único de entrada
├─ core/	Otimizadores cognitivos do meta-repositório
├─ gates/	39 portões determinísticos globais
├─ scripts/	Orquestrador síncrono e gestores (componentes, deps)
├─ componentes/	O cofre canônico: skills, comandos, specs, src-core
├─ tools/	As 8 ferramentas homologadas
├─ docs/	Protocolos, planos, manuais, relatórios, livros

A regra que governa essa topologia é a de **fonte física canônica**: um componente (habilidade, comando, hook, MCP) nasce em `componentes/<ferramenta ou compartilhado>/<tipo>/` e é distribuído para as pastas de cada assistente por `python ecossistema.py components sync`. As pastas `.claude/`, `.opencode/`, `.gemini/`, `.agents/`, `.cursor/`, `.mimocode/` e as demais são **destinos gerados**, nunca fontes. Editar um destino diretamente é um erro que o portão `G_COMPONENTE_AGNOSTICO` detecta.

3.2 A CLI unificada como barramento

`ecossistema.py` é o barramento do monorepo: 21 comandos que roteiam para as ferramentas, os gestores e o orquestrador. Ele resolve três problemas de integração que seriam invisíveis num diagrama ingênuo.

Isolamento de importação. Cada ferramenta é executada como subprocesso com `PYTHONPATH` apontando para o próprio diretório — `cmd_forge` exporta `PYTHONPATH=tools/aidd-forge`, `cmd_master` exporta `tools/aidd-master`, e assim por diante. Ferramentas com módulos de mesmo nome não colidem.

Autorrecuperação antes de qualquer importação de terceiro. Duas guardas rodam antes de `import click`: a primeira verifica Python ≥ 3.10 e exibe um banner educativo em vez de um `SyntaxError` críptico; a segunda intercepta `ImportError` de dependência e oferece `--auto-bootstrap`, que instala `requirements.txt` e tenta uma única vez mais. Máquina virgem nunca vê traceback cru.

Codificação forçada em UTF-8. `run_command()` define `PYTHONIOENCODING=utf-8` no ambiente de todo subprocesso. O comentário no código registra o motivo real: no console padrão do Windows (codepage cp1252), um único `print` com caractere fora do repertório derrubava a ferramenta inteira com `UnicodeEncodeError` antes de terminar o pipeline.

Grupo de comandos	Comandos	Destino
Ferramentas	<code>forge</code> , <code>planner</code> , <code>generate</code> , <code>factory</code> , <code>bridge</code> , <code>master</code> , <code>enterprise</code> , <code>ops</code>	Subprocesso isolado em <code>tools/<ferramenta></code>
Fluxos	<code>pure</code> , <code>open</code> , <code>bridge</code> (com flags de fluxo), <code>run-fluxo</code>	<code>scripts/orquestrador_sincrono.py</code>
Evolução	<code>melhoria</code> , <code>plan</code> , <code>orchestrate</code>	Habilidades donas das três etapas
Governança	<code>audit</code> , <code>status</code> , <code>harness</code> , <code>preflight-host</code>	<code>pre-commit</code> , portões globais, diagnóstico de host

Grupo de comandos	Comandos	Destino
Gestão	<code>components</code> , <code>dependencia</code>	<code>scripts/gestor_componentes.py</code> , <code>scripts/gestor_dependencias.py</code>

3.3 O núcleo compartilhado

`componentes/compartilhado/src-core/` contém 35 módulos que são a biblioteca comum das ferramentas e dos projetos gerados. Os principais, por família:

Família	Módulos	Papel
Contratos e erros	<code>result.py</code> , <code>cqrs.py</code> , <code>saga.py</code> , <code>circuit_breaker.py</code>	Mônada <code>Result</code> , separação comando/consulta, resiliência
Persistência	<code>database.py</code> , <code>database_adapter.py</code> , <code>transaction_log.py</code> , <code>local_first.py</code>	SQLite WAL, adaptadores, log transacional com cache LRU
Integração	<code>events.py</code> , <code>webhooks.py</code> , <code>outbox_worker.py</code> , <code>jobs.py</code> , <code>sync.py</code>	EventBus, webhooks HMAC, padrão outbox, filas
Exposição	<code>openapi.py</code> , <code>mcp_server.py</code> , <code>swagger.html</code> , <code>webhook_studio.html</code> , <code>mcp_studio.html</code>	O Quarteto Sine Qua Non materializado
Segurança	<code>security.py</code> , <code>token_revocation.py</code> , <code>package_verifier.py</code> , <code>assinatura_manifesto.py</code>	Zero-trust, revogação, verificação de pacote, assinatura Ed25519
Observabilidade	<code>metrics.py</code> , <code>opentelemetry.py</code> , <code>logs.py</code>	Métricas, tracing distribuído, log estruturado
Economia cognitiva	<code>caveman_protocol.py</code> , <code>intent_router.py</code> , <code>subagent_engine.py</code>	Compressão tri-fase, roteamento de intenção, subagentes efêmeros
Escrita segura	<code>escritor_atomico.py</code> , <code>materializador.py</code>	Gravação staging → fsync → <code>os.replace</code> , materialização com rollback
Frontend	<code>nextjs_exporter.py</code> , <code>design_catalog.py</code>	Exportação do Padrão-Ouro Next.js, catálogo de design

O portão `G_DRIFT_NUCLEO_COMPARTILHADO` compara os arquivos de núcleo compartilhados por linhagem entre `aidd-master` e `aidd-enterprise` contra o baseline em `gates/baseline_nucleo_compartilhado.json` e reprova divergência não documentada.

3.4 A arquitetura alvo dos sistemas gerados

Toda aplicação que sai do ecossistema, por qualquer fluxo, converge para a mesma arquitetura — e isso é imposto por portões, não por convenção.

Monólito modular em fatias verticais. Cada módulo de negócio é uma fatia autônoma com `router.py`, `service.py`, `repository.py`, `dtos.py` e `events.py`; o frontend espelha a estrutura com `hooks/`, `components/`, `types.ts` e `page.tsx`. Import direto entre módulos de negócio é proibido (`G_ARQUITETURA`, `G_ISOLATION_AUDIT`, `G_AST_BOUNDED_CONTEXT`).

Comunicação por evento. `JOIN SQL` entre contextos delimitados e chave estrangeira rígida entre fatias são proibidos. A comunicação atravessa o `EventBus` ou interfaces públicas de serviço.

Mônada Result. Todo método de serviço retorna `Result.ok()` ou `Result.fail()`. Exceção crua não vaza para a camada de apresentação (`G_QUALIDADE`).

Persistência segura. SQLite sempre em modo WAL; consultas sempre com marcador `?`; zero concatenação de string em SQL; exclusão sempre lógica (`deletado_em IS NULL`) (`G_SEGURANCA`).

Camadas de frontend separadas. Componentes de apresentação pura em `components/ui/` não podem fazer chamada de rede direta — nada de `fetch`, `axios` ou `ky` (`G_FRONTEND_LAYERS`).

Paridade REST/MCP. Nenhuma funcionalidade pode existir apenas via Model Context Protocol sem contrapartida REST equivalente (`G_PROTOCOL_FALLBACK`). Essa é a tradução arquitetural da Lei #6: um sistema cuja única porta é o MCP estaria preso ao ecossistema de agentes.

Quarteto Sine Qua Non. `/docs`, `/webhooks`, `/mcp` e `/docs/guia` presentes desde o nascimento, cobrindo 100% dos módulos e se atualizando a cada módulo novo. Verificado em dois níveis desde 20/09/2026: `G_CONTRACT_ROT` prova que o servidor vivo bate com o `openapi.json` commitado, e `G_QUARTETO_SINE_QUA_NON` prova que os 4 pilares realmente existem no deliverable gerado.

O QUARTETO COMO DECISÃO ARQUITETURAL, NÃO COMO ENFEITE

Os quatro estúdios existem porque o sistema gerado precisa ser operável por três públicos simultâneos: humanos (`/docs`, `/docs/guia`), sistemas externos (`/webhooks`) e agentes de IA (`/mcp`). Um sistema que só serve a um dos três volta para a mesa seis meses depois. A Lei 10 exige, além da presença, o **dinamismo**: um módulo novo aparece nos quatro sem intervenção manual.

3.5 A malha multi-harness

O ecossistema roda nativamente em dez ambientes de assistente. O mapeamento físico foi obtido por teste empírico contra instalações reais em 5 de setembro de 2026, e o registro de proveniência está na referência canônica.

Assistente	Caminho real de habilidades	Confir-mado
Claude Code / Grok	<code>.claude/skills/</code> + <code>.claude/commands/</code> + <code>CLAUDE.md</code>	Sim
OpenCode	<code>.opencode/skills/<nome>/SKILL.md</code> (v1.18.29)	Sim
MimoCode	<code>.mimocode/skills/<nome>/SKILL.md</code> (v0.1.14)	Sim
Gemini CLI	<code>.gemini/extensions/<nome>/gemini-extension.json</code>	Sim (ex-tensões)
Hermes Agent	<code>.agents/skills</code> e <code>.hermes/skills</code> (requer <code>trust</code>)	Sim
Antigravity (agy)	<code>.agents/skills/<nome>/SKILL.md</code> e <code>.agents/rules/</code>	Sim
Cursor IDE	<code>.cursor/rules/</code> (mecanismo de arquivo único)	Sim
Kiro CLI	<code>.kiro/agents/</code> — fora do padrão de habilidades daqui	N/A
Freebuff	Instalado, sem modo não interativo para validação	Parcial

O portão `G_UNIVERSAL_HARNESS` audita a paridade e o wiring de habilidades, MCPs e hooks em todos os assistentes; `G_HARNESS_COMPAT` verifica que os artefatos multi-harness da raiz permanecem sincronizados entre si.

3.6 Rastreabilidade do capítulo

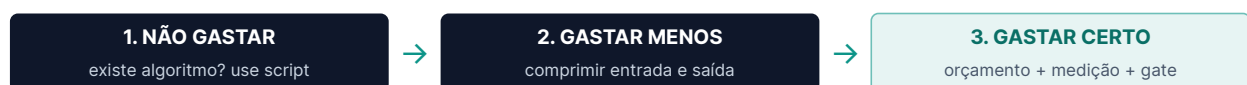
`ecossistema.py` (linhas 1–190 para as guardas e 188–380 para o roteamento); `componentes/compartilhado/src-core/`; `gates/G_DRIFT_NUCLEO_COMPARTILHADO.py`; `gates/G_UNIVERSAL_HARNESS.py`; `gates/G_PROTOCOL_FALLBACK.py`; `gates/G_FRONTEND_LAYERS.py`; `gates/G_ISOLATION_AUDIT.py`; `docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md` §5; `docs/protocolos/PADRAO-OURO-STACK-TECNOLOGICA.md`.

Capítulo 4 — Economia de tokens aplicada no ecossistema

4.1 Como foi pensada: o token como orçamento, não como efeito colateral

A economia de tokens no AIDD não é uma otimização tardia — é um critério de projeto com status de lei (Lei #4, Economia Extrema de Tokens). A tese é que o consumo de tokens é **um recurso orçado por etapa**, com valor declarado antes da execução, medido durante e auditado depois por um portão determinístico.

Disso decorre a hierarquia de decisão que atravessa todo o repositório:



4.2 Mecanismo 1 — Determinismo como economia primária

A maior economia do ecossistema não vem de comprimir texto: vem de **não chamar o modelo**. As Fases 1, 5, 6 e 7 do `aidd-generator` declaram consumo zero porque são Python puro. As Fases 1, 4, 5 e 6 do `aidd-factory` são 100% determinísticas por contrato (`G_FACTORY_DETERMINISTIC`). Todo o `aidd-forge`, todo o `aidd-master`, todo o `aidd-enterprise` e o pipeline de três fases do `aidd-ops` operam sem chamada de modelo. Os 145 portões `G_*.py` do repositório são, sem exceção, determinísticos: leem arquivo, aplicam regra, retornam código de saída.

4.3 Mecanismo 2 — O Protocolo Caveman Ultra tri-fase

`componentes/compartilhado/src-core/caveman_protocol.py` implementa um motor de compressão de três fases com objetivo declarado de reduzir a entrada em 30% a 50% na contagem de tokens BPE.

Fase	O que faz	Idioma
INPUT	Comprime regras e instruções substituindo perífrases por termos curtos	English Caveman
PROCESSING	Raciocínio interno telegráfico, de três a cinco linhas de cadeia de pensamento	English Caveman Ultra
OUTPUT	Entrega final: código completo, tipado, com <code>Result</code> , sem stub	PT-BR de padrão alto

A compressão opera por mapas de substituição progressivos em três intensidades (`LITE`, `FULL`, `ULTRA`). O nível `LITE` troca perífrases inteiras — "it is necessary to" vira "must", "conduct an analysis of" vira "analyze", "in order to" vira "to". O nível `FULL` acrescenta a remoção de artigos e verbos de ligação. A classe `CavemanProtocol` mantém uma `SessaoProtocolo` com histórico de `EstatisticasTokens` por operação — tokens originais, comprimidos, economizados, percentual de redução, intensidade, fase e tempo de compressão em milissegundos — e expõe `economia_total_percentual` acumulada da sessão.

Duas validações impedem que a compressão degrade a entrega: `_validar_idioma_output` confere que a saída final está no idioma correto e `_validar_completude_tecnica` verifica que o código entregue está completo. Comprimir a saída para o usuário não é permitido — a economia acontece na entrada e no raciocínio, nunca na qualidade do que é entregue.

O linter estático `tools/aidd-generator/scripts/core/caveman_linter.py` audita, por AST e sem gastar um único token, se as constantes de prompt das fases seguem a tríade ENTRADA (inglês) → COT (caveman) → SAÍDA (PT-BR).

4.4 Mecanismo 3 — Fatiamento dinâmico de contexto

`core/context_slicer.py` implementa o `DynamicContextSlicer`: em vez de ler arquivos inteiros, ele faz parse de AST e extrai **apenas assinaturas** — classes com seus métodos, funções com seus argumentos e número de linha, sem corpo de implementação. O método `get_minimal_context()` monta um payload compacto a partir de múltiplos arquivos. O objetivo declarado no cabeçalho do módulo é montar um payload estruturado abaixo de 150 tokens e eliminar a necessidade de busca exploratória com `grep` e de leitura de arquivo inteiro.

Essa é a contraparte em código da regra "grafo primeiro" do `AGENTS.md`: as ferramentas do MCP `code-review-graph` (`get_minimal_context_tool`, `get_review_context_tool`, `query_graph_tool`, `get_impact_radius_tool`) devem ser consultadas antes de `Grep`, `Glob` ou leitura direta.

4.5 Mecanismo 4 — Descarregamento de esquema MCP

`core/mcp_dynamic_router.py` ataca um custo fixo frequentemente ignorado: registrar dezenas de ferramentas MCP no boot do assistente consome milhares de tokens no prompt de sistema de **toda** requisição da sessão, mesmo quando nenhuma dessas ferramentas será usada.

O `MCPDynamicRouter` expõe apenas duas funções essenciais — `aidd_dispatch(tool_name, arguments)` e `aidd_query_schema(tool_name)` — mantendo o catálogo completo em um registro lazy. `list_tool_names()` devolve só identificadores; o esquema JSON completo de uma ferramenta só é materializado sob demanda estrita, quando ela vai ser realmente usada.

4.6 Mecanismo 5 — Estado em arquivo, não em conversa

O `AGENTS.md` do `aidd-generator` é explícito: o estado é persistido em `PLANO-EXECUCAO-ESTRUTURADO.json` e os agentes leem o estado em JSON — cerca de 5 mil tokens — em vez do histórico conversacional, que cresce sem limite.

`core/cognitive_ledger.py` complementa com um livro-razão imutável em SQLite WAL (`.aidd/cognitive_ledger.db`): registra intenções, decisões arquiteturais, hashes de arquivos alterados e vereditos de portões, permitindo recuperação de sessão sem depender de histórico volátil. `get_latest_session_state()` devolve os dez eventos mais recentes de uma sessão.

O `core/pipeline_state.py` do gerador adiciona retomada inteligente: com `--resume`, o pipeline pula fases já completas cujos artefatos são válidos, evitando reconsumo de tokens em trabalho já feito.

4.7 Mecanismo 6 — Orçamento formal e auditoria de tokenomics

`tools/aidd-generator/config/token_budgets.json` declara o teto por fase e o limiar de alerta de desvio (1,2 — vinte por cento acima do orçamento).

Fase	Nome	Orçamento	Natureza declarada
1	Pesquisador	15.000	Pesquisa de mercado e referências via LLM
2	Analizador	25.000	Análise crítica de requisitos e riscos via LLM
3	Designer	20.000	Design arquitetural de dados e componentes

Fase	Nome	Orçamento	Natureza declarada
4	Planejador	5.000	Decisão de escopo e planejamento determinístico
5	Criador	5.000	Scaffold e materialização determinística
6	Documentador	5.000	Geração determinística tripartite de documentação
7	Auto-crítica	5.000	Análise crítica determinística e cálculo de score
8	Implementador	50.000	Implementação funcional com loop de correção

O portão `G_TOKENOMICS` lê `_pipeline_state.json` e verifica quatro coisas: que o consumo real por fase não excede o orçamento; que a economia total é maior ou igual ao limiar configurável (padrão 30%); que **a origem da medição está rotulada honestamente** (`autodeclarado` ou `medido_api`); e que nenhuma fase afirma "medição real" para um valor autodeclarado. O último item é a Lei #8 aplicada à própria métrica de economia — o ecossistema audita a honestidade do seu próprio número de marketing.

`scripts/benchmark_tokenomics.py` fecha o ciclo com medição real via `tiktoken`, comparando o pipeline atual contra uma baseline legada (despejo bruto, loop de correção sem diff, regras monolíticas) e produzindo relatório JSON auditável com economia percentual real por fase, custo em dólares e resultado de `pytest`.

4.8 Mecanismo 7 — Compressão seletiva de prosa

`scripts/compressor_middleware.py` integra a habilidade `sandeco-token-reduce` (LLMLingua-2) sob política estrita: comprime **somente prosa descritiva** — resumos de referência, narrativa da Fase 6 — e **nunca** código, JSON de esquema, caminhos ou identificadores. Se o LLMLingua-2 não estiver disponível, o pipeline segue com fallback determinístico transparente (truncamento por seção com elipse) em vez de falhar. Toda compressão registra telemetria com a taxa real obtida.

4.9 Mecanismo 8 — Disciplina de contexto no terminal

As regras operacionais do `AGENTS.md` são, em efeito prático, mecanismos de economia: canalizar toda saída verbosa para `tail/grep`, nunca despejar lockfile ou log bruto no contexto, usar blocos exatos de busca e substituição em vez de reescrever arquivos inteiros, e responder como executor silencioso. Um único `pytest` sem filtro pode consumir mais contexto do que três fases inteiras do pipeline.

4.10 Rastreabilidade do capítulo

`componentes/compartilhado/src-core/caveman_protocol.py`; `core/context_slicer.py`; `core/mcp_dynamic_router.py`; `core/cognitive_ledger.py`; `tools/aidd-generator/config/token_budgets.json`; `tools/aidd-generator/scripts/gates/G_TOKENOMICS.py`; `tools/aidd-generator/scripts/core/caveman_linter.py`; `tools/aidd-generator/scripts/core/pipeline_state.py`; `tools/aidd-generator/scripts/compressor_middleware.py`; `tools/aidd-generator/scripts/benchmark_tokenomics.py`; `AGENTS.md` §1 e Lei #4.

Capítulo 5 — Governança executável: portões, hooks e auditoria

5.1 O que é um portão de qualidade no AIDD

Um portão (quality gate) é um script Python determinístico que lê o repositório, aplica uma regra e retorna código de saída: **0 aprova, 1 bloqueia**. Não existe resultado intermediário, não existe aviso que passa. Essa é a Lei #2 — Qualidade Binária — e é o que dá ao ecossistema a propriedade mais importante da sua governança: **a regra não depende de alguém lembrar dela**.

Todo portão obedece a três invariantes de construção: é determinístico (zero chamada de modelo), é executável isoladamente (`python gates/G_X.py`) e tem teste próprio — o diretório `gates/` contém, ao lado de cada portão relevante, o seu `test_g_*.py`.

5.2 Os 39 portões globais

Portão	O que audita
<code>G_ECOSSISTEMA_INTEGRIDADE</code>	Integridade física, sintática e estrutural dos subprojetos e das habilidades
<code>G_DRIFT_NUCLEO_COMPARTILHADO</code>	Divergência não documentada no núcleo compartilhado entre <code>aidd-master</code> e <code>aidd-enterprise</code>
<code>G_HARNESS_COMPAT</code>	Sincronia entre comandos, habilidades e arquivos-ponteiro multi-harness da raiz
<code>G_UNIVERSAL_HARNESS</code>	Paridade e wiring de habilidades, MCPs e hooks em todos os assistentes
<code>G_COMPONENTE_AGNOSTICO</code>	Cobertura multi-harness de todo componente novo ou modificado contra o manifesto
<code>G_SEGREDOS</code>	Credencial hardcoded em todo arquivo rastreado pelo git (delega ao <code>detect-secrets</code>)
<code>G_CLI_HELP_CONSISTENCIA</code>	Por AST: flags citadas em <code>print()/raise()</code> contra flags realmente definidas em <code>add_argument</code>
<code>G_ZERO_HEADLESS</code>	Execução de subagentes headless paralelos; garante o modo interativo como rota primária
<code>G_INFRA_COMPOSE</code>	Integridade, sintaxe e segurança de orquestrações Docker Compose (Checkov + PyYAML)
<code>G_HADOLINT</code>	Boas práticas OCI, segurança e sintaxe de todos os Dockerfiles
<code>G_TESTES_REAIS</code>	Roda <code>pytest</code> de verdade em cada <code>tools/<ferramenta></code> ; falha se qualquer suíte tiver <code>failed > 0</code>
<code>G_DEPENDENCIAS_PIN_HASH</code>	Pin exato (<code>==</code>), hash SHA-256 por pacote nos lockfiles e <code>pip install --require-hashes</code> no CI
<code>G_HONESTIDADE_ROTULO</code>	Lei #8: termos de marketing proibidos em <code>print()/raise()</code> dos scripts de portão
<code>G_ARQUITETURA_DELIVERABLE</code>	Conformidade com Clean Architecture/DDD via AST
<code>G_ESCRITOR_ATOMICO</code>	Uso de gravação atômica nos arquivos críticos do ecossistema
<code>G_TRANSACTION_LOG_LRU</code>	Por AST + SHA-256: fonte única, destinos byte-idênticos, símbolos críticos e testes espelhados
<code>G_SUPPLY_CHAIN</code>	Conformidade e integridade criptográfica da cadeia de suprimentos

Portão	O que audita
G_FRONTEND_LAYERS	Chamada de rede direta dentro de componentes de apresentação pura (<code>components/ui/</code>)
G_ISOLATION_AUDIT	Por AST: imports diretos entre fatias verticais
G_PROTOCOL_FALLBACK	Paridade REST vs MCP: nada exposto só via MCP sem contrapartida OpenAPI
G_LLM_PROMPT_SHIELD	Por AST: clientes de LLM usam <code>PromptShield</code> contra injeção e jailbreak
G_DRIFT_ANALYZER	Duplicidade e redundância estrutural de funções entre fatias, para extração ao núcleo
G_PROTOTYPE_REWRITE	Isolamento de <code>sandbox/</code> e proibição de promover protótipo a <code>src/</code> sem suíte TDD espelhada
G_ORQUESTRADOR_SINCRONO	Integridade do orquestrador da tríade, da CLI <code>run-fluxo</code> e dos esquemas de hand-off
G_DOCS_ROT	Documentação viva, links quebrados e planos fora dos buckets canônicos
G_PORTAO_PROVA_QUE_MORDE	Meta-portão (Lei #13): bloqueia gate novo/alterado sem teste que prove reprovação real (exit 1)
G_LEI_DECLARA_PORTAO	Meta-portão (Lei #8): bloqueia Lei Inviolável sem portão ou "sem gate" declarado em <code>AGENTS.md</code>
G_CONTRACT_ROT	Rotas e formatos do servidor em execução contra o <code>openapi.json</code> commitado (Lei #10)
G_ENV_ROT	Por AST: leituras de variável de ambiente sem entrada correspondente em <code>.env.example</code> (Lei #9)
G_SKILL_ROT	Resolução estática de path/script/comando citado em <code>SKILL.md</code> (Lei #9)
G_MIGRATION_ROT	Migração de banco aplica/reverte/reaplica sem divergência num SQLite efêmero (Lei #3)
G_IDIOMA_LEI_4	Densidade de prosa em PT-BR nos caminhos que devem ser inglês compacto (tickets, skills, núcleo)
G_LIVRO_EVIDENCIA	Rastreabilidade e evidência de um livro-texto gerado — roda sob demanda, não a cada commit deste repositório
G_DETERMINISMO_LEI_1	Por AST: SDK de LLM conhecido importado em rota declarada mecânica (Lei #1)
G_SAIDA_BINARIA	Por AST: todo portão em <code>gates/</code> sai estritamente via <code>sys.exit(0)</code> ou <code>sys.exit(1)</code> (Lei #2)
G ESTRUTURA_ESTADO	Artefatos de estado do orquestrador (<code>flight_plan.json</code> , logs <code>.jsonl</code>) contra o formato esperado (Lei #3)
G_DISCIPLINA_TESTE_FERRAMENTA	Alteração em <code>tools/<ferramenta>/</code> sem relatório contemporâneo em <code>docs/teste-end-to-end/</code> (Lei #9)
G_QUARTETO_SINE_QUA_NON	Presença real dos 4 pilares (<code>/docs/</code> , <code>/webhooks</code> , <code>/mcp</code> , <code>/docs/guia</code>) num deliverable gerado (Lei #10)
G_STACK_PADRAO_OURO	Dependências de frontend/backend gerado contra o padrão-ouro, com respeito a override explícito (Lei #11)

5.3 A execução: `pre-commit` como runner

`python ecossistema.py audit` delega a `pre-commit run --all-files`. A configuração em `.pre-commit-config.yaml` registra 34 hooks, todos **locais** (`repo: local`, `language: system`) — o que torna a auditoria hermética, offline e independente de assistente ou sistema operacional, em conformidade com a Lei #6.

As regras de gatilho são três: `always_run: true` roda em todo commit; `files: ^tools/` roda apenas quando um arquivo correspondente está no stage (é o caso de `G_TESTES_REAIS`, que roda pytest de verdade); e `stages: [manual]` roda apenas sob demanda explícita.

ZERO PORTÕES PENDENTES POR CAUSA RAZ DESCONHECIDA — O ESTADO HONESTO EM 20/09/2026

Em 19/09/2026 dois portões estavam em `stages: [manual]`: **G_SEGREDOS**, movido em 2026-09-08 por uma inconsistência real entre execução direta (aprovara) e via hook (reprovava) cuja causa raiz não tinha sido identificada; e **G_ARQUITETURA_DELIVERABLE**, por 18 violações legadas conhecidas em `src/core`. Os dois foram fechados: a causa raiz do primeiro foi encontrada (o `detect-secrets` exige o baseline staged e a lista completa de arquivos rastreados no scan com merge) e os 31 alertas foram triados individualmente como falsos positivos; as 18 violações do segundo foram corrigidas movendo as chamadas SQL para a camada de infraestrutura. Ambos voltaram a `always_run: true` e aprovam com exit 0.

O único portão que ainda roda em `stages: [manual]` é **G_LIVRO_EVIDENCIA** — por desenho, não por pendência: ele audita o livro-texto de um projeto **gerado** pelo ecossistema, não este repositório, então não há sentido em rodá-lo a cada commit daqui.

5.4 Hooks: os três níveis de automação

O ecossistema usa hooks em três camadas distintas, e confundi-las gera erro de diagnóstico.

Hooks de git (`.github/hooks/pre-commit`, instalados pelo `aidd-forge` via `core/git_hooks.py`): interceptam o commit e executam os portões. É a camada que transforma a Lei #2 em bloqueio real.

Hooks de assistente (`.claude/settings.json` e equivalentes): reagem a eventos do assistente. No Claude Code há dois configurados: em `SessionStart`, consulta o status do grafo de conhecimento; em `PostToolUse` com matcher `Edit|Write`, atualiza o grafo incrementalmente (`code-review-graph update --skip-flows`) com timeout de 30 segundos. Ambos degradam silenciosamente quando a ferramenta não está instalada — `command -v code-review-graph >/dev/null 2>&1 || exit 0` — o que preserva o agnosticismo.

Hooks canônicos compartilhados (`componentes/compartilhado/hooks/`): `crg_session_start.py`, `crg_update.py` e `regra10_check.py`, com invólucros `.sh` e `.cmd` para funcionar em POSIX e Windows. São a fonte física distribuída aos assistentes pelo `components sync`.

5.5 Gestão de dependências e cadeia de suprimentos

`scripts/gestor_dependencias.py`, exposto como `python ecossistema.py dependencia`, mantém o registro de habilidades e MCPs de terceiros com quatro subcomandos: `bootstrap` (instala o que falta), `add-skill`, `add-mcp`, `list` e `verify`. O `CLAUDE.md` do projeto instrui o agente a rodar `dependencia verify` silenciosamente no início da sessão e, em caso de falha, executar `bootstrap` e reportar o resultado em uma frase — sem pedir que o usuário digite o comando.

No eixo da cadeia de suprimentos, `G_DEPENDENCIAS_PIN_HASH` exige pin exato em `requirements.txt` e `requirements-dev.txt`, hash SHA-256 completo por pacote nos lockfiles (`requirements.lock`, `requirements-dev.lock`) e `pip install --require-hashes` no CI. `G_SUPPLY_CHAIN` audita a integridade criptográfica, e `componentes/compartilhado/src-core/package_verifier.py` faz a verificação em tempo de execução.

5.6 Rastreabilidade do capítulo

`gates/` (39 portões + suítes de teste); `.pre-commit-config.yaml` (linhas 1–66 para a documentação das decisões, 67–220 para os hooks); `.claude/settings.json`; `componentes/compartilhado/hooks/`; `scripts/gestor_dependencias.py`; `docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md` §4.

PARTE II — MESO: OS FLUXOS

A Parte II desce um nível. Enquanto a Parte I descreveu o organismo, esta descreve os seus **movimentos**: as esteiras que levam uma intenção até um sistema em produção.

São quatro movimentos. Três deles são fluxos de **criação** — a Tríade Canônica [aidd-pure](#), [aidd-open](#) e [aidd-bridge](#) — e diferem entre si apenas na estratégia de construção do artefato central. O quarto é o fluxo de **evolução**, [/melhoria](#) → [/plan](#) → [/orchestrate](#), que atua sobre o que já existe.

O capítulo 6 estabelece a anatomia comum, porque as três criações compartilham princípio, cabeça e cauda. Os capítulos 7, 8 e 9 tratam de cada fluxo pelo mesmo gabarito de sete itens usado no restante do livro. O capítulo 10 trata da evolução.

Capítulo 6 — Anatomia comum de um fluxo de criação

6.1 O que é um fluxo no AIDD

Um fluxo é uma **esteira síncrona de sete etapas**, executada por `scripts/orquestrador_sincrono.py`, em que cada etapa valida formalmente o contrato de handoff antes de passar a bola para a seguinte. Se qualquer etapa falhar, o pipeline é abortado imediatamente com código de saída 1 — não existe continuação com dado fabricado nem etapa pulada silenciosamente.



A **cabeça** (FORGE → PLANNER) e a **cauda** (MASTER → ENTERPRISE → OPS → AUDITORIA) são idênticas nos três fluxos. A única diferença é a etapa 3, o motor:

Fluxo e nome canônico	Motor da etapa 3	Acionamento	Estratégia de construção
Fluxo 01 — <code>aidd-pure</code>	<code>aidd-generator</code>	<code>/pure · ecossistema.py pure</code>	Do zero puro: código autoral sob medida, com TDD
Fluxo 02 — <code>aidd-open</code>	<code>aidd-factory</code>	<code>/open · ecossistema.py open</code>	Motores open-source curados e integrados
Fluxo 03 — <code>aidd-bridge</code>	<code>aidd-bridge</code>	<code>/bridge · ecossistema.py bridge</code>	Desacoplamento de low-code e migração para PostgreSQL

Esse desenho — chamado no `AGENTS.md` de **Universal Convergence Funnel** — significa que a decisão "como construir" é tomada uma única vez, no início, e não contamina o resto da esteira. A entrega final é a mesma forma arquitetural nos três casos.

6.2 Engenharia agêntica no nível do fluxo

Quem decide o quê

Etapa	Regime	Quem decide
1 — Forge	Determinístico	Script: cria estrutura, injeta governança, instala hooks
2 — Planner	Misto	Usuário no PRÉ-PLANO interativo; script valida e exporta
3 — Engine	Varia	Fluxo 01 usa modelo em 4 das 8 fases; Fluxos 02 e 03 são quase determinísticos
4 — Master	Determinístico	Script: scaffold de fatia vertical e frontend
5 — Enterprise	Determinístico	Script: injeção com SHA-256 e verificação de drift
6 — Ops	Determinístico	Script: sizing, compose, validação de manifesto
7 — Auditoria	Determinístico	Script: manifesto de execução

A concentração do regime de julgamento na etapa 3 é deliberada: é a única etapa em que não existe resposta certa computável, porque é ali que se decide o que construir e não apenas como materializar o que já foi decidido.

Regras que o fluxo segue

O orquestrador aplica quatro regras invariantes, independentemente do fluxo: execução **estritamente sequencial** (Lei #7 — nunca há duas etapas correndo em paralelo em segundo plano); **abortar na primeira falha**; **validar esquema antes do handoff**; e **gravar artefato em disco** a cada fronteira, de modo que a execução seja retomável e auditável depois.

6.3 Arquitetura no nível do fluxo: os artefatos de fronteira

Cada etapa deixa um arquivo na pasta do projeto. Esses arquivos são o registro permanente do que aconteceu — e são eles, não a conversa, que a etapa seguinte lê.

Artefato	Quem escreve	Quem lê	Conteúdo
<code>.git/</code> , governança, hooks	Forge	Todas	Repositório, <code>AGENTS.md</code> , portões, hooks de commit
<code>PLANNER.json</code>	Planner	Orquestrador	Plano BDD/SDD completo do projeto
<code>HANDOFF_PLANNER_ENGINE.json</code>	Orquestrador	Motor da etapa 3	Contrato validado contra <code>handoff-planner-to-engine.schema.json</code>
<code>HANDOFF_ENGINE_MASTER.json</code>	Orquestrador	aidd-master	Fatias geradas, artefatos de frontend, testes executados
<code>HANDOFF_MASTER_ENTERPRISE.json</code>	Orquestrador	aidd-enterprise	Diretório, estado do servidor, rotas do Quarteto, componentes
<code>HANDOFF_ENTERPRISE_OPS.json</code>	Orquestrador	aidd-ops	Auditoria SHA-256, drift verificado, manifesto de deploy
<code>ORQUESTRAÇÃO_EXECUÇÃO.json</code>	Orquestrador	Humano/auditoria	Manifesto final com fluxo, nome, slug, domínio, timestamp, etapas

ESTADO HONESTO DO ORQUESTRADOR SÍNCRONO

O orquestrador valida cada payload de handoff contra o esquema JSON formal antes de gravá-lo — essa parte é real e bloqueia de fato. Porém, na versão atual de `scripts/orquestrador_sincrono.py`, **parte dos campos dos payloads é montada pelo próprio orquestrador com valores fixos** em vez de ser lida da saída real da ferramenta: `testes_executados` recebe um bloco constante, `sha256_audit_ok` recebe `True` literal e o manifesto final grava `status: "CONFORME_100_POR_CENTO"` de forma fixa.

Em termos práticos: hoje o orquestrador garante que o **formato** do contrato está correto e que cada ferramenta **saiu com código 0**, mas o conteúdo de alguns campos de telemetria ainda não reflete medição real. Este livro registra o fato porque a Lei 8 (Honestidade de Rótulo) se aplica à documentação; a checagem forte de cada etapa continua existindo, mas ela vem do código de saída da ferramenta e dos portões próprios dela, não desses campos.

6.4 Economia de tokens no nível do fluxo

O desenho do fluxo é, por si só, o maior mecanismo de economia do ecossistema. Três propriedades explicam por quê.

Cinco das sete etapas custam zero tokens. Forge, Master, Enterprise, Ops e Auditoria são inteiramente determinísticas. O custo de um fluxo completo se concentra nas etapas 2 (parcialmente) e 3.

O contrato substitui a memória. Como cada etapa lê um JSON tipado, não há necessidade de manter histórico conversacional entre etapas. Um agente que retoma o fluxo na etapa 5 lê aproximadamente 5 mil tokens de estado, não o transcript inteiro.

O motor é escolhido pelo custo, não só pela adequação. Os Fluxos 02 e 03 são quase totalmente determinísticos porque curam software que já existe em vez de escrevê-lo. Quando o problema admite reuso, o fluxo correspondente custa uma fração do Fluxo 01.

Fluxo	Onde o token é gasto	Ordem de grandeza da etapa 3
01	Fases 2, 3, 4 e 8 do gerador (analisar, desenhar, planejar, implementar)	Dezenas de milhares
02	Fases 2, 3 e 7 do factory (curadoria e integração), com portões AST + bandit	Baixo
03	Nenhuma fase do pipeline de seis etapas exige modelo	Próximo de zero

6.5 Como disparar um fluxo

Pelo assistente, basta o slash command. Pelo terminal:

```
# Fluxo 01 – do zero puro
python ecossistema.py pure --nome "Clínica Vida" --slug clinica --dominio saude --pasta ../proj_clinica

# Fluxo 02 – motores open-source
python ecossistema.py open --nome "Central CRM" --slug crm --dominio vendas --pasta ../proj_crm

# Fluxo 03 – libertação de low-code
python ecossistema.py freedom --nome "App Lovable" --slug app --dominio geral \
  --pasta ../proj_app --origem ../export-lovable

# Forma longa equivalente, com simulação segura
python ecossistema.py run-fluxo --fluxo pure --nome "..." --slug ... --dominio ... --pasta ... --dry-run
python ecossistema.py run-fluxo --fluxo freedom --nome "..." --slug ... --dominio ... --pasta ... --origem ...
```

O `--dry-run` executa toda a lógica de decisão e validação de contrato sem escrever no disco nem tocar em git — é o modo recomendado para conhecer o fluxo antes de rodá-lo de verdade.

6.6 Rastreabilidade do capítulo

`scripts/orquestrador_sincrono.py` (etapas 1 a 7, mapa de fluxos, validação de esquema); `componentes/compartilhado/specs/*.schema.json`; `AGENTS.md` §3; `gates/G_ORQUESTRADOR_SINCRONO.py`; `ecossistema.py` (`cmd_pure`, `cmd_open`, `cmd_aidd_bridge`, `cmd_run_fluxo`).

Capítulo 7 — Fluxo 01: aidd-pure (do zero puro)

7.1 O que é

O Fluxo 01 constrói o sistema **do zero, sob medida**, sem reaproveitar base de código existente. É o fluxo de maior custo em tokens e o de maior liberdade arquitetural: o motor `aidd-generator` executa oito fases que vão de pesquisa de referências reais até implementação de código funcional verificada por `pytest`.

É o fluxo indicado quando o domínio é específico o bastante para que nenhum motor open-source sirva, ou quando o controle total sobre a arquitetura é requisito.

7.2 Passo a passo de execução

Disparo	<code>/pure ou python ecossistema.py pure --nome ... --slug ... --dominio ... --pasta ...</code>
Motor da etapa 3	<code>aidd-generator</code> — pipeline de 8 fases
Persistência alvo	SQLite em modo WAL
Frontend alvo	Next.js + TypeScript + Tailwind (Padrão-Ouro, Lei 11)
Entrega	Sistema completo, testado, blindado e com manifesto de deploy

1. **Forge** cria a pasta, inicializa git e roda `forge init --force`, instalando governança, portões locais e hooks de pre-commit.
2. **Planner** roda `planner init --fluxo 1`, produz `PLANNER.json` e o orquestrador valida e grava `HANDOFF_PLANNER_ENGINE.json`.
3. **Generator** roda `generate "<nome>: sistema para <domínio>" --pasta <p> --implementar-codigo`. As oito fases executam na ordem 1→2→3→4→5→8→6→7 (a Fase 8 entra logo após a 5 para que a documentação da Fase 6 descreva o código real e a auto-crítica da Fase 7 audite o sistema funcional).
4. **Master** roda `master init <slug>` e `master add-module <slug>`, harmonizando o resultado em fatia vertical com frontend espelhado.
5. **Enterprise** roda `enterprise inject rule regra-integridade-<slug>` e `enterprise verificar-drift`.
6. **Ops** valida a presença de `Dockerfile` e `docker-compose.yml` no projeto.
7. **Auditoria** grava `ORQUESTRACAO_EXECUCAO.json`.

7.3 Portões de qualidade do fluxo

Além dos portões globais, o Fluxo 01 acumula os portões do gerador: transição de fase bloqueada por `python scripts/validar_fase.py --fase N` (saída 1 impede progressão), mais `G_TOKENOMICS`, `G_CYBERSECURITY_OWASP`, `G_SANDBOX_NIVEL_1`, `G_SESSAO_HERMETICA`, `G_BLOQUEAR_SEGREDOS`, `G_INTEGRACAO_CROSS_SCRIPT`, `G_VERIFICAR_LLM_PRONTO`, `G_INJECT` e `G_HARNESS_COMPAT`.

7.4 Determinismo e economia de tokens

Quatro das oito fases não gastam token: 1 (Pesquisador), 5 (Criador), 6 (Documentador) e 7 (Auto-crítica). O orçamento total declarado das fases que gastam é de 115 mil tokens, dos quais 50 mil são

da Fase 8 (Implementador) — que é onde o código de fato nasce. A Fase 4 (Planejador) é híbrida: 50% determinística no modo automático e interativa quando o usuário participa.

7.5 O que entrega, como e para quem

Entrega um sistema completo em `<pasta>`: backend Python modular em fatias verticais, frontend Next.js, banco SQLite WAL inicializado, suíte pytest executada de verdade, documentação tripartite (HTML, Markdown e PDF via Typst), relatório de auto-crítica com score em seis dimensões e roadmap, e o Quarteto Sine Qua Non ativo. Entrega **para o** `aidd-master` dentro do fluxo, e **para o desenvolvedor** como produto final.

Capítulo 8 — Fluxo 02: `aidd-open` (motores open-source)

8.1 O que é

O Fluxo 02 **não escreve o sistema: integra sistemas que já existem**. O motor `aidd-factory` recebe um plano de infraestrutura resolvido, cura os motores open-source adequados ao nicho, gera a camada de integração (gateway/BFF, compose, inicialização de banco, variáveis de ambiente) e entrega uma aplicação multi-serviço.

É o fluxo indicado quando o domínio já tem software consolidado — CRM, atendimento, agendamento, automação — e o valor está na integração, não na reimplementação. É a aplicação direta da política anti-NIH (not invented here) do ecossistema.

8.2 Passo a passo de execução

A cabeça e a cauda são idênticas às do Fluxo 01. A etapa 3 roda `factory curate --dominio <d> --output <pasta>/factory_output`.

O contrato de entrada do factory é **único e fechado**: ele consome apenas `PLANO-INFRAESTRUTURA.json`, validado contra `componentes/compartilhado/specs/plano-infraestrutura.schema.json`, e nunca lê catálogos de nicho diretamente — o plano já chega resolvido. O contrato de saída também é único: `FACTORY_OUTPUT.json`, listando todos os artefatos gerados com status, que o orquestrador de deploy consome.

O nicho dinâmico

O catálogo fixo tem cinco nichos (`clinicas`, `delivery`, `farmacias`, `b2b_industrial`, `energia_solar`). Para domínios fora desse catálogo, existe o caminho do **nicho dinâmico**: `fase_1_intake.saida.nicho_slug` recebe o prefixo `dinamico_`, e `01_analisador.py` sintetiza os blocos e bancos lógicos a partir da própria lista de ferramentas do plano (baseline `traefik` mais `postgres` condicional), em vez de exigir que `templates/infra/nichos/<slug>.json` exista.

A razão dessa decisão está registrada no `AGENTS.md` do `aidd-planner`: pela Lei #7 (Desenvolvedor no Controle), a lista de ferramentas curada pelo usuário no PRÉ-PLANO é autoritativa — não um palpite por palavra-chave. Por isso `exportar_para_fluxo_factory` reutiliza `pipeline_ops.montar_plano_em_memoria(..., ferramentas_planejadas=...)` em vez de tentar casar o texto com um dos cinco nichos fixos.

8.3 Portões, determinismo e economia

O `AGENTS.md` do factory é explícito sobre o regime de cada fase:

Fases	Regime	Portão associado
1, 4, 5, 6	100% determinístico, zero LLM	<code>G_FACTORY_DETERMINISTIC</code>
2, 3, 7	LLM com portões AST + bandit	<code>G_FACTORY_INPUT</code> , <code>G_FACTORY_OUTPUT</code>

O resultado é que o Fluxo 02 custa uma fração do Fluxo 01 em tokens: o trabalho pesado — escrever o software — já foi feito por terceiros, e o que resta é integração, que é majoritariamente mecânica.

8.4 O que entrega, como e para quem

Entrega uma stack multi-serviço integrada: gateway/BFF, compose unificado com os serviços curados, banco inicializado, `.env` resolvido, integração entre os serviços e `FACTORY_OUTPUT.json` como manifesto auditável. Entrega **para o `aidd-master`** (que fatia a integração em VSA) e, no fim da esteira, **para o `aidd-ops`**, que provisiona a infraestrutura correspondente.

Capítulo 9 — Fluxo 03: `aidd-freedom` (motor `aidd-bridge` — libertação de low-code)

9.1 O que é

O Fluxo 03 resolve um problema específico e muito concreto: uma aplicação foi gerada em uma plataforma low-code (Lovable, v0, Bolt), funciona, o usuário gosta da interface — e está presa ao fornecedor, com banco Supabase, autenticação proprietária e custo recorrente. O motor `aidd-bridge` extrai essa aplicação, converte o banco para PostgreSQL puro com emulação PostgREST, empacota em Docker e a coloca em VPS própria **preservando a interface**.

9.2 Passo a passo de execução

O pipeline determinístico do bridge tem seis fases, documentadas no cabeçalho de `aidd_bridge/pipeline_bridge.py`:



1. **Scan e análise de componentes** (`LovableScanner`): mapeia páginas, componentes shadcn e migrações.
2. **Desacoplamento de banco** (`DataBridge`): sanitiza o SQL do Supabase para PostgreSQL puro com emulação PostgREST, de modo que o cliente `@supabase/supabase-js` do frontend continue funcionando **sem refatoração**.
3. **Separação de camadas e frontend** (`FrontendLiberator`): produz `.env.production` limpo, sem credencial de fornecedor.
4. **Empacotamento DevOps OCI** (`DevOpsPackager`): Dockerfile com usuário não-root, Nginx com cabeçalhos OWASP, compose.
5. **Conector VSA e Quarteto Sine Qua Non**: exporta `/docs`, `/webhooks`, `/mcp` e `/docs/guia` em `quarteto_sine_qua_non/` para harmonização no `aidd-master`.
6. **Portões dedicados**: `G_BRIDGE_VENDOR_LOCKIN`, `G_BRIDGE_DOCKER_OCI`, `G_BRIDGE_POSTGRESQL` e `G_BRIDGE_VSA_COMPAT`.

9.3 As quatro operações sensíveis

O bridge lida com dados reais de produção, e por isso quatro operações têm tratamento especial.

Migração de autenticação (`migrate-auth`): preserva os hashes de senha de `auth.users` ao migrar contas para o ambiente self-hosted — as pessoas continuam entrando com a mesma senha. A operação é **preview por padrão** e só escreve com `--apply`.

Fusão multi-aplicação (`merge`): combina várias aplicações em fatias verticais segregadas em `src/modules/<app>/` com roteamento unificado.

DNS Cloudflare (`cloudflare_dns.py`): cria e remove registros durante o empacotamento e o teardown.

Teardown atômico (`destroy`): limpeza completa de stacks Docker Swarm, volumes e registros DNS — **exige confirmação** a menos que `--yes` seja passado explicitamente.

9.4 Determinismo e economia

Nenhuma das seis fases do pipeline exige chamada de modelo. O Fluxo 03 é o mais barato dos três em tokens, porque a tarefa é de transformação estrutural — parse, reescrita de SQL, reempacotamento — e transformação estrutural tem algoritmo.

9.5 O que entrega, como e para quem

Entrega o projeto libertado: código React/Vite/Tailwind preservado, banco PostgreSQL puro com PostgREST, Dockerfile e compose prontos para VPS, `.env.production` sem credencial de fornecedor, e o Quarteto exportado. Entrega **para o `aidd-master`** dentro do fluxo e **para a VPS do usuário** ao final, via `aidd-ops`.

Capítulo 10 — O fluxo de evolução: /melhoria → /plan → /orchestrate

10.1 Por que existe um quarto fluxo

Os três fluxos canônicos criam. Este evolui. A diferença é essencial: criar tem estado inicial vazio e, portanto, nenhuma restrição de compatibilidade; evoluir tem um sistema em funcionamento, com usuários e dados, e uma pergunta que precede qualquer código — o que exatamente está errado, e como sabemos disso?

O ecossistema responde com uma esteira de três etapas, cada uma com uma **parada obrigatória** para decisão humana.

Etapa	Comando	Entra	Sai	Parada no fim
1	/melhoria	Pedido em linguagem natural, ou plano a reanalisar	Relatório em docs/melhorias/ com Nota Atual + evidência	"gero o plano a partir disto?"
2	/plan	Relatório da etapa 1, ou pedido direto	Pasta em docs/planos/<nome>/, tudo em rascunho	"aprova este plano?"
3	/orchestrate	Plano aprovado	Execução real das frentes	Escolha de ambiente + Plano de Voo

10.2 Etapa 1 — /melhoria: nota com evidência

A skill dona investiga o **código real** na ordem grafo → Grep/Read → reprodução real, atribui uma **Nota Atual de 0 a 10 com evidência** e gera relatório .html e .json em docs/melhorias/. Ela também reanalisa um plano existente, comparando previsto contra implementado item a item.

A palavra-chave é evidência. Uma memória de projeto registra a regra com clareza: todo caso de auditoria exige script ou comando próprio, executado do zero — leitura de código nunca substitui teste. E o código de saída precisa ser real: `cmd | tail` devolve o código do `tail`, não o do comando; a forma correta é redirecionar para arquivo e capturar `$?` na mesma linha.

10.3 Etapa 2 — /plan: rascunho que não se aprova sozinho

A skill dona (`plan`, com motor `planos-auditoria-runner`) gera a estruturação padronizada e os rascunhos de planos de auditoria, evolução ou teste, com checagem determinística de cercas markdown. Duas proibições são explícitas: **não fabricar decisões** e **não fabricar aprovações**.

APROVAR O PLANO NÃO É AUTORIZAR A EXECUÇÃO

O ecossistema separa rigorosamente os dois atos. Dizer "aprovado" sobre um plano aprova o **documento**; não é sinal verde para implementar. A autorização de execução é uma instrução separada, dada depois — e é exatamente isso que a parada obrigatória no fim da etapa 2 protege.

10.4 Etapa 3 — /orchestrate: roteador de ambiente e Plano de Voo

A skill dona é um **roteador de ambiente**: decide entre o aplicativo ORCA, subagentes ou git worktree nativo, monta um Plano de Voo e executa as frentes do plano. Não é o comando do AIDD Ops — esse é /ops.

As regras fixas da via ORCA estão gravadas na referência canônica e nasceram de incidentes reais: carregar o manual da versão instalada (`orca skills get orca-cli`) antes de qualquer comando; usar mesa independente (`--no-parent`) por padrão, mesa filha só a pedido explícito; **nunca** lançar o assistente com `--resume <id-de-sessão>`; esperar `tui-idle` com `satisfied: true` antes de enviar o prompt; e nunca reenviar no silêncio.

`--dry-run` gera e imprime o Plano de Voo sem tocar em git nem criar agente.

10.5 Um comando, um dono

A regra de propriedade descrita no capítulo 2 vale integralmente aqui: /melhoria, /plan e /orchestrate têm cada um exatamente uma skill dona; `planos-auditoria-runner` e `orca-plan-orchestrator` são motores acionados pelas donas, sem slash command próprio. A justificativa está registrada: duas skills respondendo ao mesmo comando com regras opostas causou o incidente de mesas recursivas de 11 de setembro de 2026.

10.6 Rastreabilidade do capítulo

`docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md` §3 (tabela das três etapas e regras da via ORCA); `componentes/compartilhado/skills/melhoria/`, `plan/`, `orchestrate/`, `planos-auditoria-runner/`, `orca-plan-orchestrator/`; `ecossistema.py` (`cmd_melhoria`, `cmd_plan`, `cmd_orchestrate`); `docs/melhorias/`; `docs/planos/`.

PARTE III — MICRO: AS OITO FERRAMENTAS

Esta é a parte mais longa do livro, e a mais repetitiva por desenho. Cada uma das oito ferramentas é descrita pelo mesmo gabarito, na mesma ordem, para que o leitor possa comparar duas ferramentas lendo a mesma seção de cada capítulo.

O gabarito tem oito seções. As três primeiras situam a ferramenta: o que ela é, qual é o seu papel dentro do fluxo e qual é o seu papel dentro do ecossistema. A quarta descreve como foi pensada, como está estruturada e como está configurada. As três seguintes são a análise em camadas — funcionamento **individual**, funcionamento **dentro do fluxo** e funcionamento **dentro do ecossistema** — e cada uma delas responde sempre aos mesmos sete itens:

OS SETE ITENS DE CADA CAMADA

1. **Passo a passo de execução** — o que acontece, em ordem.
2. **Portões de qualidade** — o que bloqueia, e com qual critério.
3. **Habilidades** — quais skills operam ou são operadas nessa camada.
4. **Determinismo** — quais scripts fazem o trabalho sem chamar modelo.
5. **Ferramentas acessadas** — CLIs, MCPs, APIs e binários externos.
6. **Hooks e regras** — o que dispara automaticamente e quais leis se aplicam.
7. **Entrega** — o que entrega, como entrega e para quem entrega.

A oitava seção é a Rastreabilidade: os arquivos do repositório que sustentam o capítulo.

A ordem dos capítulos segue a ordem da esteira, não a ordem alfabética: primeiro a cabeça ([aidd-forge](#), [aidd-planner](#)), depois os três motores ([aidd-generator](#), [aidd-factory](#), [aidd-bridge](#)), depois a cauda ([aidd-master](#), [aidd-enterprise](#), [aidd-ops](#)).

Capítulo 11 — aidd-forge: a fundação

Papel	Bootstrap determinístico, isolamento de micro-ambiente, fatiamento de fase e purga de contexto
Etapa na esteira	1 de 7 — primeira etapa dos três fluxos
Slash command	/forge [caminho] ou /aidd-init
CLI	python ecossistema.py forge init inject audit conform
Regime	100% determinístico — zero chamada de modelo
Padrão de erro	Mônada Result.ok() / Result.fail() em toda fronteira

11.1 O que é a ferramenta

aidd-forge é o **injetor de governança** do ecossistema. Ele pega um diretório — vazio ou com um projeto existente — e o transforma em um ambiente onde as leis do AIDD são executáveis: cria a estrutura canônica, materializa os arquivos de regra para todos os assistentes, instala os portões locais, instala os hooks de git e prepara os micro-ambientes de fase.

A analogia do README.md é precisa: é a fundação e o chassi. Antes dele, o diretório é um lugar onde a IA pode fazer o que quiser. Depois dele, é um lugar onde a IA não consegue commitar código que viole as regras, porque o hook de pre-commit roda os portões e bloqueia.

11.2 O papel da ferramenta dentro do FLUXO

Dentro de qualquer um dos três fluxos, aidd-forge é a **etapa 1**, e a sua função é garantir que as seis etapas seguintes rodem sobre terreno preparado. O orquestrador síncrono cria a pasta, garante git init e então executa ecossistema.py forge init <pasta> --force. Se essa etapa falhar, o fluxo aborta imediatamente — não existe caminho que pule a fundação.

A ordem importa: sem o forge, o aidd-planner escreveria PLANNER.json num diretório sem governança, e o código gerado pelo motor da etapa 3 poderia ser commitado sem passar por portão nenhum.

11.3 O papel da ferramenta dentro do ECOSSISTEMA

Fora do contexto de um fluxo, o aidd-forge tem três funções permanentes.

É o **injetor universal de componentes**: forge inject <tipo> <nome> materializa deterministicamente dez tipos de artefato — skill, mcp, rule, spec, roteiro, config, command, hook, sub-agent e script — com rollback atômico via materializador.py. É assim que uma habilidade nova nasce no ecossistema.

É o **auditor de conformidade de governança**: forge audit verifica se um projeto está aderente às regras, e forge conform aplica correções automáticas.

É o **sincronizador multi-harness**: core/harness_sync.py e core/harness_manifest.py mantêm a paridade entre os dez ambientes de assistente, e o core/agents_md_anchor.py garante que o AGENTS.md permaneça a âncora canônica com os arquivos-ponteiro dos demais assistentes apontando para ele.

11.4 Como foi pensada, está estruturada e configurada

A tese

A tese do forge é que **governança que depende de alguém lembrar não é governança**. Por isso ele não escreve documentação pedindo boas práticas: ele instala mecanismos que tornam a violação impossível ou detectável. O `AGENTS.md` do forge lista cinco invariantes: Zero Stubs, mônada `Result` em toda fronteira de serviço, isolamento por purga de contexto, injetor universal com rollback atômico, e sete portões determinísticos obrigatórios.

A estrutura

O pacote `aidd_forge/` tem quatro subdiretórios — `commands/`, `core/`, `schemas/`, `templates/` — e o núcleo concentra 23 módulos com responsabilidade estreita:

Módulo do núcleo	Responsabilidade
<code>injector.py</code>	Injeção da infraestrutura AIDD no projeto alvo
<code>universal_injector.py</code>	Injeção de componentes por tipo, com perfis
<code>injector_profiles.py</code>	Os dez tipos suportados e o destino físico de cada um
<code>materializador.py</code>	Escrita com transação e rollback atômico
<code>injection_schema.py</code>	Validação do contrato de injeção
<code>audit_engine.py</code> / <code>audit_checks.py</code> / <code>audit_report.py</code>	Motor, checagens e relatório de auditoria
<code>conform_engine.py</code> / <code>conform_fixers.py</code>	Correção automática de não conformidade
<code>harness_sync.py</code> / <code>harness_manifest.py</code>	Distribuição e manifesto multi-harness
<code>agents_md_anchor.py</code>	<code>AGENTS.md</code> como âncora canônica
<code>git_hooks.py</code>	Instalação dos hooks de pre-commit
<code>phase_fencer.py</code>	Cercas de fase: cada fase enxerga apenas o seu micro-ambiente
<code>subagent_purger.py</code>	Purga de contexto de subagente após validação AST
<code>sandbox_worktree.py</code>	Isolamento por worktree para experimentos
<code>token_optimizer.py</code>	Otimização de contexto no material injetado
<code>context_linter.py</code>	Lint do contexto injetado
<code>camada_detector.py</code> / <code>detector.py</code>	Deteção de camada arquitetural e de ferramentas do host
<code>orca_bridge.py</code>	Integração com o ambiente de orquestração ORCA

A configuração

Os sete portões que o forge instala no projeto alvo vivem em `aidd_forge/templates/gates/: G_BLOQUEAR_SEGREDOS, G_CONTRACTS, G_CYBERSECURITY_OWASP, G ESTRUTURA_AST, G_HARNESS_COMPAT, G_INJECT, G_PERFORMANCE e G_TESTES_REAIS`.

O `ecossistema.py` adiciona uma camada de autorreparo: antes de rodar qualquer comando do forge, `_reparar_instalacao_editable_aidd_forge()` verifica se `pip show aidd-forge` aponta para este clone e, se apontar para um caminho morto — uma worktree deletada, uma pasta renomeada —, reinstala silenciosamente com `pip install --no-deps -e`. O comentário no código é explícito sobre o escopo: isso não é necessário para `python ecossistema.py forge`, que já roda isolado por `PYTHONPATH`; existe para que chamadas diretas de `python -m aidd_forge.cli` e os testes da própria ferramenta continuem funcionando após um clone novo.

11.5 Como funciona individualmente

Passo a passo. `forge init <caminho> [--force]` executa: detecção do estado do diretório alvo → materialização da estrutura AIDD → injeção do `AGENTS.md` âncora e dos arquivos-ponteiro por assistente → cópia dos sete portões-template → instalação dos hooks de git → criação das cercas de fase → relatório de status em uma linha. Toda escrita passa pelo `materializador.py`, que mantém transação: se qualquer passo falhar, o estado anterior é restaurado.

`forge inject <tipo> <nome>` resolve o perfil do tipo em `injector_profiles.py`, calcula o destino físico com `resolve_destination()`, valida contra `injection_schema.py`, materializa e sincroniza para os assistentes.

`forge audit <caminho> [--fmt] [--output]` roda o motor de auditoria e emite relatório. `forge conform <caminho> [--dry-run] [--items N]` aplica os corretores automáticos.

Portões. Os sete portões-template, mais o portão global `G_INJECT` e `G_COMPONENTE_AGNOSTICO` quando a injeção alcança o ecossistema.

Habilidades. `aidd-forge-runner` é a skill dona de `/forge.aidd-componentes` e `aidd-skills` operam sobre o resultado da injeção.

Determinismo. Integral. Não há chamada de modelo em nenhum caminho do forge — é Python puro operando sobre templates, AST e sistema de arquivos.

Ferramentas acessadas. `git` (init, instalação de hooks), `pip` (autorreparo da instalação editável), sistema de arquivos. Nenhuma API externa.

Hooks e regras. O forge é a ferramenta que **instala** hooks — `.githooks/pre-commit` via `core/git_hooks.py`. As regras que segue: Zero Stubs, mônada `Result`, purga de contexto, rollback atômico e os sete portões.

Entrega. Entrega um diretório governado. O formato da entrega é o próprio sistema de arquivos do projeto alvo. O destinatário é a etapa seguinte do fluxo (o `aidd-planner`) ou, fora de fluxo, o desenvolvedor que rodou `/forge` num projeto existente.

11.6 Como funciona dentro da camada FLUXO

Passo a passo. O orquestrador cria a pasta, roda `git init` se necessário, e chama `forge init <pasta> --force`. Código de saída diferente de zero aborta o fluxo inteiro.

Portões. No contexto do fluxo, o forge não apenas roda os seus portões: ele **instala** os portões que todas as etapas seguintes terão de satisfazer. É a etapa que arma o resto da esteira.

Habilidades. `fluxo-01-runner`, `fluxo-02-runner` e `fluxo-03-runner` invocam o forge como primeira etapa de cada fluxo.

Determinismo. 100%. É uma das cinco etapas de custo zero em tokens.

Ferramentas acessadas. As mesmas do modo individual, mais o `ecossistema.py` como invocador.

Hooks e regras. A partir do fim da etapa 1, o hook de pre-commit está ativo no projeto — o que significa que qualquer commit feito pelas etapas 3 a 7 passa pelos portões.

Entrega. Entrega ao `aidd-planner` um diretório com git inicializado, governança instalada e hooks ativos.

11.7 Como funciona dentro da camada ECOSSISTEMA

Passo a passo. Fora de fluxo, o forge é acionado para criar componentes novos (`inject`), auditar projetos (`audit`) e corrigir divergência (`conform`). A criação de uma habilidade nova segue o caminho: `forge inject skill <nome>` → o componente nasce em `componentes/compartilhado/skills/<nome>/` →

`python ecossistema.py components sync --tipo skills` distribui para os dez assistentes → `G_COMPONENTE_AGNOSTICO` audita a cobertura.

Portões. `G_COMPONENTE_AGNOSTICO`, `G_UNIVERSAL_HARNESS`, `G_HARNESS_COMPAT` e `G_INJECT` são os portões do ecossistema que julgam o trabalho do forge.

Habilidades. `aidd-skills` (criação de habilidade), `aidd-componentes` (gestão de componentes), `aidd-mcp` (criação de servidor MCP), `skill-creator-runner`.

Determinismo. Integral.

Ferramentas acessadas. `scripts/gestor_componentes.py` via `ecossistema.py components`.

Hooks e regras. A regra da **fonte física canônica**: o componente nasce em `componentes/`, nunca numa pasta de assistente. Editar `.claude/skills/<nome>` direto é o erro que `G_COMPONENTE_AGNOSTICO` detecta.

Entrega. Entrega componentes canônicos distribuídos e projetos conformes. O destinatário é o ecossistema inteiro — toda ferramenta e todo assistente consome o resultado.

11.8 Rastreabilidade

`tools/aidd-forge/AGENTS.md`; `tools/aidd-forge/aidd_forge/cli.py`; `tools/aidd-forge/aidd_forge/core/` (23 módulos); `tools/aidd-forge/aidd_forge/templates/gates/` (7 portões); `ecossistema.py::cmd_forge` e `reparar_instalacao_editable_aidd_forge`; `scripts/orquestrador_sincrono.py::etapa_01_forge`.

Capítulo 12 — aidd-planner: o plano formal

Papel	Motor de planejamento, intake SDD/BDD e geração do combustível dos três fluxos
Etapa na esteira	2 de 7
Slash command	/planner
CLI	python ecossistema.py planner init validate export audit
Artefato central	PLANNER.json, validado contra schemas/planner_schema.json
Portões próprios	G_PLANNER_SCHEMA, G_PLANNER_SINE_QUA_NON, G_PLANNER_COERENCIA_FLUXO

12.1 O que é a ferramenta

`aidd-planner` é o **intake formal** do ecossistema: transforma uma conversa sobre o que se quer construir em um documento tipado que os motores conseguem executar. O `AGENTS.md` o define como "motor de planejamento, intake SDD/BDD e geração de combustível para a Tríade Canônica".

A entrada é um PRÉ-PLANO interativo conduzido com o usuário. A saída é `PLANNER.json`: entidades, endpoints, cenários BDD, módulos funcionais, arquitetura alvo e a configuração explícita do Quarteto Sine Qua Non.

12.2 O papel da ferramenta dentro do FLUXO

É a etapa 2, e é a **única etapa da cabeça em que o usuário participa da decisão**. O que o planner decide aqui governa tudo o que vem depois: o campo `meta.fluxo_alvo` determina a estrutura do `payload_especifico_fluxo`, e é esse payload que o motor da etapa 3 consome.

O orquestrador roda `planner init --fluxo N --nome --slug --dominio --pasta`, lê o `PLANNER.json` resultante, monta o payload de handoff, valida contra `handoff-planner-to-engine.schema.json` e grava `HANDOFF_PLANNER_ENGINE.json`.

12.3 O papel da ferramenta dentro do ECOSSISTEMA

No ecossistema, o planner é o **guardião da Lei #10**: nenhum plano passa sem `/docs`, `/webhooks`, `/mcp` e `/docs/guia` marcados como `ativo: true`. O portão `G_PLANNER_SINE_QUA_NON` (local, em `tools/aidd-planner/gates/`) reprova qualquer plano que tente nascer sem o Quarteto; desde 20/09/2026 ele é complementado pelo `gates/G_QUARTETO_SINE_QUA_NON.py` na raiz, que audita a presença real dos 4 pilares no deliverable já gerado, não só na intenção declarada no plano.

É também o ponto onde a Lei #7 (Desenvolvedor no Controle) tem a sua expressão mais forte: a lista de ferramentas curada pelo usuário no PRÉ-PLANO é **autoritativa**, e o ecossistema mudou o próprio código para respeitá-la.

O ACHADO DE 18/09/2026 — POR QUE O EXPORT MUDOU

O `AGENTS.md` do planner registra um achado real com data: o `export --formato factory` precisa devolver **exatamente** o envelope `fase_1_intake / fase_2_curadoria / fase_3_sizing` que o `contrato_factory.py` valida contra `plano-infraestrutura.schema.json` — nunca um formato próprio.

A implementação (`exportar_para_fluxo_factory` em `src/core/planner_engine.py`) reutiliza `pipeline_ops.montar_plano_em_memoria(texto,`

`ferramentas_planejadas=payload_especifico_fluxo.ferramentas_opensource`), que é o caminho do "nicho dinâmico" do `aidd-ops`, em vez de tentar casar o pedido com um dos cinco nichos fixos do catálogo. A justificativa é a Lei 7: a lista curada pelo usuário vale mais que um palpite por palavra-chave.

12.4 Como foi pensada, está estruturada e configurada

A ferramenta é enxuta por desenho: `src/cli.py` com quatro subcomandos e `src/core/planner_engine.py` mais `src/core/design_system.py` como núcleo. O contrato é o `schemas/planner_schema.json`, e a validação é feita por três portões dedicados em `gates/`.

Os quatro invariantes arquiteturais declarados no `AGENTS.md` são: **Zero Stubs** (nunca produzir `TODO`, `FIXME` ou marcador de espaço em `PLANNER.json` — todas as entidades, endpoints e cenários BDD devem ser completos e tipados); **Quarteto Sine Qua Non** com os quatro ativos; **polimorfismo estrito** governado por `meta.fluxo_alvo`; e **qualidade binária** contra os três portões.

12.5 Como funciona individualmente

Passo a passo.

```
python ecossistema.py planner init --fluxo 1 --nome "Clinica Vida" --slug clinica --dominio saude --
pasta ../proj
python ecossistema.py planner validate ../proj/PLANNER.json
python ecossistema.py planner export ../proj/PLANNER.json --formato factory
python ecossistema.py planner audit ../proj
```

`init` conduz o intake e gera o `PLANNER.json` canônico; `validate` confere conformidade contra o esquema; `export` converte para o formato do fluxo de destino; `audit` roda os três portões.

Portões. `G_PLANNER_SCHEMA` (conformidade estrutural), `G_PLANNER_SINE_QUA_NON` (os quatro estúdios ativos) e `G_PLANNER_COERENCIA_FLUXO` (coerência entre `meta.fluxo_alvo` e o `payload_especifico_fluxo`). Os três precisam sair com 0.

Habilidades. `aidd-planner-runner` é a dona de `/planner`. As habilidades procedimentais `/aidd-grill` e `/aidd-spec` alimentam o intake: a primeira extrai invariantes e casos de borda por entrevista socrática, a segunda converte o resultado em especificação com critérios binários de aceite.

Determinismo. A geração, validação e exportação são determinísticas. O que não é determinístico é o **diálogo** do PRÉ-PLANO — mas ali quem decide é o usuário, não o modelo.

Ferramentas acessadas. `jsonschema` para validação; `pipeline_ops` do `aidd-ops` para o caminho do nicho dinâmico no export.

Hooks e regras. Lei #10 (Quarteto), Lei #7 (usuário autoritativo), Zero Stubs.

Entrega. Entrega `PLANNER.json` no diretório do projeto. Entrega **para o motor do fluxo escolhido** — `aidd-generator`, `aidd-factory` OU `aidd-bridge`.

12.6 Como funciona dentro da camada FLUXO

Passo a passo. Etapa 2 de 7. O orquestrador invoca, lê o resultado, monta e valida o handoff. Falha em qualquer ponto aborta o fluxo.

Portões. Os três próprios, mais a validação do orquestrador contra `handoff-planner-to-engine.schema.json`, cujos seis campos obrigatórios são `versao_schema`, `fluxo_alvo`, `metadados_projeto`, `quarteto_sine_qua_non`, `arquitetura_alvo` e `modulos_funcionais`.

Habilidades. `fluxo-01-runner`, `fluxo-02-runner`, `fluxo-03-runner`.

Determinismo. A parte automatizada é integral; o intake é interativo por desenho.

Ferramentas acessadas. `ecossistema.py planner` como invocador.

Hooks e regras. O campo `arquitetura_alvo` do handoff carrega o Padrão-Ouro (Lei #11): `padrao_frontend: nextjs_typescript_tailwind`, `padrao_backend: fastapi_modular_vsa`, e a persistência que varia por fluxo — `sqlite_wal` nos Fluxos 01 e 02, `postgresql` no Fluxo 03.

Entrega. Entrega `HANDOFF_PLANNER_ENGINE.json` validado, para a etapa 3.

12.7 Como funciona dentro da camada ECOSSISTEMA

Passo a passo. Fora de um fluxo completo, o planner é usado para produzir e auditar planos isoladamente — por exemplo, para avaliar a viabilidade de um domínio antes de comprometer a execução.

Portões. Os três portões do planner são parte da bateria auditada por `G_ECOSSISTEMA_INTEGRIDADE`.

Habilidades. `/aidd-grill`, `/aidd-grill-docs`, `/aidd-spec`, `/aidd-tickets` — a cadeia anti-vibe coding completa desemboca num plano.

Determinismo. Validação e exportação determinísticas.

Ferramentas acessadas. `componentes/compartilhado/specs/plano-infraestrutura.schema.json` como contrato compartilhado com `aidd-ops` e `aidd-factory`.

Hooks e regras. A regra de que o export do factory deve devolver o envelope exato do `aidd-ops`, e nunca um formato próprio.

Entrega. Entrega um contrato formal que qualquer motor do ecossistema consegue consumir sem tradução. É o que torna possível trocar o motor sem reescrever o plano.

12.8 Rastreabilidade

```
tools/aidd-planner/AGENTS.md; tools/aidd-planner/src/cli.py; tools/aidd-planner/src/core/planner_
engine.py; tools/aidd-planner/schemas/planner_schema.json; tools/aidd-planner/gates/; componentes/
compartilhado/specs/handoff-planner-to-engine.schema.json; scripts/orquestrador_sincrono.py::etapa_
02_planner.
```

Capítulo 13 — `aidd-generator`: a fábrica autônoma de oito fases

Papel	Fábrica autônoma: ideia → sistema completo, testado e auditado
Etapa na esteira	3 de 7 — motor do Fluxo 01 (<code>`aidd-pure`</code>)
Slash command	<code>/generate <ideia></code>
CLI	<code>python ecossistema.py generate "<ideia>" --pasta <p> [--implementar-codigo]</code>
Estado	<code>PLANO-EXECUCAO-ESTRUTURADO.json</code> + <code>.aidd/cache/_pipeline_state.json</code>
Orçamento total declarado	115.000 tokens nas 4 fases que usam modelo

13.1 O que é a ferramenta

`aidd-generator` é a ferramenta mais complexa do ecossistema e a única que constrói software do zero. É um pipeline de oito fases, com máquina de estados formal validada por JSON Schema Draft 2020-12, que vai da pesquisa de referências reais até a entrega de código funcional verificado por `pytest`.

13.2 O papel da ferramenta dentro do FLUXO

É o motor da etapa 3 do Fluxo 01. Recebe o handoff do planner, executa as oito fases e entrega um sistema que o `aidd-master` vai harmonizar em fatias verticais.

O orquestrador o invoca com `--implementar-codigo`, o que altera a ordem de execução para 1→2→3→4→5→**8**→6→7. O motivo é documentado no próprio pipeline: rodar a Fase 8 logo após a 5 faz com que a documentação da Fase 6 descreva o código real e a auto-crítica da Fase 7 audite o projeto funcional completo, em vez de auditar um esqueleto.

13.3 O papel da ferramenta dentro do ECOSSISTEMA

No ecossistema, o gerador cumpre dois papéis além de gerar software.

É o **laboratório de tokenomics**: `config/token_budgets.json`, `G_TOKENOMICS`, `benchmark_tokenomics.py`, `compressor_middleware.py`, `caveman_linter.py` e `core/pipeline_state.py` estão todos aqui. As técnicas de economia de tokens do ecossistema são desenvolvidas e medidas neste contexto.

É o **laboratório de engenharia agêntica**: Fleet Discovery (descoberta automática dos assistentes instalados no host), Context-Purge Engine (subagentes efêmeros), Intent Router (detecção de intenção), micro-ambientes por fase e o protocolo delegado nasceram aqui.

13.4 Como foi pensada, está estruturada e configurada

As oito fases

ase	Nome	Regime	Orçamento	Saída principal
1	Pesquisador	Determinístico	0	<code>insights_phase1.json</code> — referências reais

ase	Nome	Regime	Orçamento	Saída principal
2	Analizador	LLM	25.000	analise_phase2.json
3	Designer	LLM (5 subagentes)	20.000	design_aidd_phase3.json
4	Planejador	Híbrido	5.000	config_global_local_phase4.json
5	Criador	Determinístico	0	Projeto em disco, git, SQLite
6	Documentador	Determinístico	0	HTML + Markdown + PDF (Typst)
7	Auto-crítica	Determinístico	0	relatorio_final.md com score
8	Implementador	LLM + loop de correção	50.000	Código funcional verificado por pytest

Micro-ambiente: a fase como unidade isolada

Cada fase tem um [AGENTS.md](#) próprio em [scripts/phases/phase_NN_<nome>/](#) declarando escopo, restrições, regras de negócio, portões, MCPs acessíveis, saída e consumo de tokens com justificativa. Apenas o micro-ambiente da fase em execução é carregado em memória.

Os portões de fase são nomeados por letra e número, e são específicos:

ase	Portões
1	R1 URLs válidas; R2 atividade recente (pushed_at < 2 anos); R3 estrutura mínima; R4 ≥ 3 referências
2	A1 esquema válido; A2 zero alucinação; A3 dados completos; A4 sem TODO/pass
3	D1 camadas completas; D2 scripts com pseudocódigo; D3 determinismo mínimo ≥ 65%
4	C1 decisões GLOBAL/LOCAL válidas; C2 symlinks resolvem para caminho real; C3 configuração completa
5	E1 arquivos em disco; E2 git init + commit; E3 SQLite inicializado; E4 permissões; S1 segurança

DUAS REGRAS QUE REVELAM A FILOSOFIA INTEIRA

A Fase 1 proíbe explicitamente o fallback de dados de teste: **"Dados REAIS apenas — fallback DADOS_TESTE é PROIBIDO"**. A Fase 2 proíbe fabricar citação: as referências citadas devem vir da Fase 1, e sem referências a análise deve ser **honesta sobre as lacunas**. O portão D3 da Fase 3 exige determinismo mínimo de 65% no design produzido — o gerador é obrigado a projetar sistemas que, eles próprios, minimizem o uso de modelo.

A máquina de estados

[scripts/fsm_engine.py](#) implementa o [GeneratorFSMEngine](#), que valida o artefato de saída de cada fase contra JSON Schema Draft 2020-12 **antes** de permitir a transição para a fase N+1. [scripts/core/pipeline_state.py](#) grava atomicamente o estado em [.aidd/cache/_pipeline_state.json](#) com validação estrita e implementa `--resume`: retomada que pula fases completas com artefatos válidos, sem reconsumo de tokens.

Os retries por fase são configurados individualmente — de uma tentativa na Fase 4 até três nas Fases 2, 3, 6 e 8, com atrasos de 2 a 5 segundos.

O Padrão-Ouro imposto no design

O micro-ambiente da Fase 3 exige que toda aplicação Web/API projetada inclua obrigatoriamente: CRUD completo com ciclo fechado; frontend no padrão Impeccable (paleta Zinc, tipografia tabular, microinterações de 150 ms, zero alertas nativos do sistema operacional via [share/ui_dialogs.js](#)); Swagger em tema escuro nativo; Studio MCP com JSON-RPC 2.0; e Studio de Webhooks com HMAC SHA-256 assíncrono e auditoria de disparos.

13.5 Como funciona individualmente

Passo a passo.

```
python scripts/preflight_llm.py # 0 = pronto, 1 = falta configuração
python ecossistema.py generate "app de habitos" --pasta ../MEU-PROJETO --implementar-codigo #
`implementação`
python scripts/gates/G_TOKENOMICS.py --pasta ../MEU-PROJETO
```

O pré-voo verifica se há `LLM_MODEL` configurado e credencial do provedor antes de gastar tempo com a Fase 1 — se faltar, o pipeline para imediatamente com mensagem clara. Sem fallback silencioso: se qualquer fase retornar `None`, o pipeline para e reporta exatamente qual fase falhou e por quê.

Portões. Os portões de fase (R, A, D, C, E, S), mais os portões da ferramenta: `G_TOKENOMICS`, `G_CYBERSECURITY_OWASP`, `G_SANDBOX_NIVEL_1`, `G_SESSAO_HERMETICA`, `G_BLOQUEAR_SEGREDOS`, `G_INTEGRACAO_CROSS_SCRIPT`, `G_VERIFICAR_LLM_PRONTO`, `G_INJECT`, `G_HARNESS_COMPAT`, e o comparativo `AUDITAR_COMPARATIVO_HARNESS`.

Habilidades. `aidd-generator-runner` é a dona de `/generate`. As Fases 1 e 2 utilizam `/aidd-spec` e `/aidd-tickets` explicitamente, conforme o `AGENTS.md`, para impedir vibe coding.

Determinismo. Quatro das oito fases são de custo zero. A validação AST é mecânica em toda escrita: na Fase 5, todo código gerado passa por `ast.parse()` antes de ir para o disco — se falhar, o arquivo **não** é criado e o erro é registrado no índice. Na Fase 8, `_validar_contrato_ast` roda antes do pytest real.

Ferramentas acessadas. API do GitHub e do HuggingFace (Fase 1, via `requests`); provedor de LLM configurado, sempre através de `solicitar_llm()` de `utils_delegacao.py` — nunca chamada direta a `litellm`; `pytest` (Fase 8); `Repomix` via `core/repomix_runner.py` para empacotamento de contexto; `Typst` (Fase 6, geração de PDF); MCP de sistema de arquivos; MCP verificador de CVE em `mcps/`.

Hooks e regras. Prompt em inglês com saída em PT-BR (tríade Caveman, auditada pelo `caveman_linter.py`); UTF-8 explícito na escrita; loop de correção com traceback real do pytest reenviado ao modelo por até três tentativas.

Entrega. Entrega o projeto completo em `<pasta>`, mais a documentação tripartite em `output/<nome>/documentos/` e o `relatorio_final.md` com score em seis dimensões (pesos 20+20+25+10+15+10), roadmap filtrado pelo score atingido e investimento estimado com premissas rotuladas explicitamente como estimativa.

13.6 Como funciona dentro da camada FLUXO

Passo a passo. Etapa 3 do Fluxo 01, invocada com `--implementar-codigo`, ordem 1→2→3→4→5→8→6→7.

Portões. Todos os portões de fase, mais a validação do handoff `handoff-engine-to-master.schema.json`.

Habilidades. `aidd-pure`, `fluxo-01-runner`.

Determinismo. No contexto do fluxo, o gerador é a única etapa com consumo significativo de tokens — as outras seis são determinísticas.

Ferramentas acessadas. As mesmas, mais o `PLANNER.json` como entrada consumida nativamente.

Hooks e regras. O hook de pre-commit instalado pelo forge na etapa 1 está ativo: o commit que a Fase 5 executa passa pelos portões.

Entrega. Entrega ao `aidd-master` um projeto com fatias, artefatos de frontend e resultado de testes.

13.7 Como funciona dentro da camada ECOSSISTEMA

Passo a passo. Fora do fluxo completo, `/generate` é o caminho mais curto entre uma ideia e um sistema. É também o campo de prova das técnicas de economia: rodar `benchmark_tokenomics.py` mede a economia real por fase contra a baseline legada.

Portões. `G_TOKENOMICS` é o portão que liga esta ferramenta à Lei #8 — ele reprova quem alegar medição real para valor autodeclarado.

Habilidades. `sandeco-token-reduce` (LLMLingua-2) via `compressor_middleware.py`; `aidd-orca` e `aidd-orchestrate` quando a execução acontece em ambiente orquestrado.

Determinismo. Metade do pipeline.

Ferramentas acessadas. O MCP `code-review-graph` para consulta de grafo antes de busca textual; `detector.py` detecta silenciosamente quais assistentes estão instalados no host (claude, codex, agy, cursor, ollama) usando apenas `shutil.which()` e `os.environ`, sem dependência externa.

Hooks e regras. Lei #1 (determinismo primeiro), Lei #4 (economia extrema), Lei #5 (zero stubs), Lei #8 (honestidade de rótulo).

Entrega. Entrega sistemas e, para o ecossistema, entrega **métricas auditáveis de economia de tokens** — o relatório JSON do benchmark com economia percentual real por fase, custo em dólares e resultado de pytest.

13.8 Rastreabilidade

`tools/aidd-generator/AGENTS.md` e `AGENTS-WORKFLOW.md`; `tools/aidd-generator/scripts/pipeline_completo.py`; `tools/aidd-generator/scripts/phases/` (8 micro-ambientes + 8 módulos de fase); `tools/aidd-generator/scripts/fsm_engine.py`; `tools/aidd-generator/scripts/core/pipeline_state.py`; `tools/aidd-generator/config/token_budgets.json`; `tools/aidd-generator/scripts/gates/` (10 portões).

Capítulo 14 — `aidd-factory`: o integrador multi-serviço

Papel	Gerador de aplicação e integração para stacks multi-serviço
Etapla na esteira	3 de 7 — motor do Fluxo 02 (<code>`aidd-open`</code>)
Slash command	<code>/factory</code> [requisito]
CLI	<code>python ecossistema.py factory --plano <arquivo> --pasta <destino></code>
Contrato de entrada	<code>PLANO-INFRAESTRUTURA.json</code> (único, fechado)
Contrato de saída	<code>FACTORY_OUTPUT.json</code> (único, fechado)

14.1 O que é a ferramenta

`aidd-factory` é o **integrador**: em vez de escrever o sistema, ele compõe uma aplicação a partir de motores open-source curados, gerando a camada que os une — gateway/BFF, orquestração compose, inicialização de banco, variáveis de ambiente e código de integração.

É a materialização da política anti-NIH do ecossistema: quando existe software consolidado que resolve o problema, escrever de novo é desperdício de tokens, de tempo e de confiabilidade.

14.2 O papel da ferramenta dentro do FLUXO

É o motor da etapa 3 do Fluxo 02. Recebe um plano já resolvido e produz os artefatos que o `aidd-master` vai fatiar em VSA e que o `aidd-ops` vai provisionar.

A restrição de entrada é dura e está no portão `G_FACTORY_INPUT`: a fábrica consome **apenas** `PLANO-INFRAESTRUTURA.json` validado contra `componentes/compartilhado/specs/plano-infraestrutura.schema.json`, e **nunca** lê catálogos de nicho diretamente — o plano já chega resolvido.

14.3 O papel da ferramenta dentro do ECOSSISTEMA

No ecossistema, a `aidd-factory` é o par simétrico do `aidd-ops`: o ops decide qual infraestrutura, a factory decide qual código de integração. Os dois compartilham o mesmo esquema de plano, o que permite que o planner exporte para qualquer um dos dois sem tradução.

É também a ferramenta que demonstra a doutrina de regimes mistos: quatro fases inteiramente determinísticas convivendo com três fases de modelo cercadas por portões de AST e `bandit`.

14.4 Como foi pensada, está estruturada e configurada

O `AGENTS.md` declara cinco invariantes: contrato de entrada único (`G_FACTORY_INPUT`), contrato de saída único (`G_FACTORY_OUTPUT`), fases determinísticas explicitamente demarcadas (`G_FACTORY_DETERMINISTIC` para as fases 1, 4, 5 e 6), Zero Stubs, e **reuso obrigatório do núcleo compartilhado** — `result.py` e `escritor_atomico.py` vêm de `componentes/compartilhado/src-core/`, não são reimplementados.

A estrutura reflete isso: `scripts/contrato_factory.py` guarda os contratos, `scripts/pipeline_factory.py` orquestra, e `scripts/phases/` contém os módulos de fase (`01_analisador.py`, `04_compose.py`, `05_init_db.py`, `06_env.py`, `09_integracao.py`). `templates/`, `schemas/`, `data/` e `gates/` completam.

O nicho dinâmico

`01_analisador.py` detecta o prefixo `dinamico_` em `fase_1_intake.saida.nicho_slug` e, nesse caso, sintetiza `blocos` e `bancos_logicos` a partir da própria lista de `ferramentas` do plano — baseline `traefik` mais `postgres` condicional — em vez de exigir que `templates/infra/nichos/<slug>.json` exista.

O `AGENTS.md` é honesto sobre o alcance dessa solução: ela é "o subconjunto determinístico do gap do Discovery Engine documentado em `docs/features/v2_arquitetura-aid-ops-factory.md` §7.1/§9.1 — um motor completo de busca no GitHub permanece trabalho futuro".

14.5 Como funciona individualmente

Passo a passo. `pipeline_factory.py` valida o plano de entrada contra o esquema → Fase 1 analisa e resolve blocos e bancos lógicos → Fases 2 e 3 usam modelo para a curadoria e o desenho de integração, com portões AST e `bandit` → Fase 4 gera o compose → Fase 5 inicializa o banco → Fase 6 resolve o `.env` → Fase 9 gera o código de integração → grava `FACTORY_OUTPUT.json` com todos os artefatos e seus status.

Portões. `G_FACTORY_INPUT`, `G_FACTORY_OUTPUT`, `G_FACTORY_DETERMINISTIC`, mais AST e `bandit` nas fases de modelo.

Habilidades. `aidd-factory-runner` é a dona de `/factory`.

Determinismo. Fases 1, 4, 5 e 6 são 100% determinísticas por contrato auditado.

Ferramentas acessadas. Docker Compose (geração de manifesto), PostgreSQL (inicialização), `bandit` (análise estática de segurança), núcleo compartilhado.

Hooks e regras. Zero Stubs — todos os geradores devem produzir código real, funcional, sem marcador de espaço nem `TODO`.

Entrega. Entrega `FACTORY_OUTPUT.json` e os artefatos listados nele. O destinatário declarado no `AGENTS.md` é o `DeployOrchestrator`.

14.6 Como funciona dentro da camada FLUXO

Passo a passo. Etapa 3 do Fluxo 02, invocada pelo orquestrador como `factory curate --dominio <d> --output <pasta>/factory_output`.

Portões. Os três próprios, mais a validação do handoff Engine → Master.

Habilidades. `aidd-open`, `fluxo-02-runner`.

Determinismo. Majoritário — é o que torna o Fluxo 02 barato em tokens.

Ferramentas acessadas. O plano exportado pelo `aidd-planner` via `exportar_para_fluxo_factory`.

Hooks e regras. A regra do contrato único: se o plano não validar contra o esquema, a fábrica não roda. Não existe modo tolerante.

Entrega. Entrega ao `aidd-master` a stack integrada para fatiamento em VSA.

14.7 Como funciona dentro da camada ECOSSISTEMA

Passo a passo. Fora do fluxo, a factory pode ser usada isoladamente para gerar a camada de integração de uma stack já planejada.

Portões. `G_INFRA_COMPOSE` e `G_HADOLINT` do ecossistema auditam os manifestos que ela produz.

Habilidades. `aidd-dependencias` (quando a curadoria exige registrar dependência nova), `aidd-componentes`.

Determinismo. Majoritário.

Ferramentas acessadas. `plano-infraestrutura.schema.json` como contrato compartilhado com `aidd-ops` e `aidd-planner`.

Hooks e regras. A política anti-NIH: reusar `result.py` e `escritor_atomico.py` do núcleo em vez de reimplementar. O portão `G_DRIFT_NUCLEO_COMPARTILHADO` detecta divergência.

Entrega. Entrega ao ecossistema a prova de que integração é majoritariamente mecânica — e, portanto, majoritariamente determinística.

14.8 Rastreabilidade

```
tools/aidd-factory/AGENTS.md;    tools/aidd-factory/scripts/contrato_factory.py;    tools/aidd-factory/
scripts/pipeline_factory.py;      tools/aidd-factory/scripts/phases/;      tools/aidd-factory/gates/;
componentes/compartilhado/specs/plano-infraestrutura.schema.json; docs/features/v2_arquitetura-aidd-
ops-factory.md.
```

Capítulo 15 — aidd-bridge: o libertador de low-code

Papel	Extrator de projetos low-code, unificador PostgreSQL e empacotador para VPS própria
Etapa na esteira	3 de 7 — motor do Fluxo 03 (`aidd-bridge`)
Slash command	<code>/bridge [comando]</code>
CLI	<code>python ecossistema.py bridge unpack scan convert-db merge pack migrate-auth destroy</code>
Origem suportada	React + Vite + Tailwind + Supabase (Lovable, v0, Bolt)
Destino	PostgreSQL puro + PostgREST + Docker em VPS própria

15.1 O que é a ferramenta

`aidd-bridge` resolve o aprisionamento por fornecedor em aplicações geradas por plataformas low-code. Ele ingere um projeto React + Vite + Tailwind + Supabase, extrai páginas, componentes shadcn e migrações, converte o banco para PostgreSQL puro com emulação PostgREST, empacota em Docker com boas práticas OCI e entrega uma aplicação que roda em servidor próprio — **sem refatorar o frontend**.

A propriedade que torna isso viável é a emulação PostgREST: o cliente `@supabase/supabase-js` do frontend continua funcionando contra o novo backend, e por isso a interface é preservada exatamente como estava.

15.2 O papel da ferramenta dentro do FLUXO

É o motor da etapa 3 do Fluxo 03. A diferença essencial em relação aos outros dois motores é a origem: aqui existe um artefato de partida real — o export da plataforma low-code — e o trabalho é de transformação, não de criação.

Ao final, exporta o Quarteto Sine Qua Non em `quarteto_sine_qua_non/` para que o `aidd-master` o harmonize na estrutura VSA canônica.

15.3 O papel da ferramenta dentro do ECOSSISTEMA

No ecossistema, o bridge é a **prova de agnosticidade aplicada a terceiros**: a Lei #6 proíbe aprisionamento no próprio ecossistema, e o bridge estende esse princípio aos projetos dos usuários, desfazendo o aprisionamento criado por outras plataformas.

É também a ferramenta com o maior número de operações destrutivas, e por isso a que tem o protocolo de confirmação mais rígido.

15.4 Como foi pensada, está estruturada e configurada

O pacote `aidd_bridge/` organiza-se por responsabilidade de transformação:

Módulo	Responsabilidade
<code>scanner.py</code>	<code>LovableScanner</code> — mapeia páginas, componentes e migrações
<code>vendor_patterns.py</code>	Catálogo de padrões proprietários a detectar e neutralizar
<code>data_bridge.py</code>	Sanitização SQL Supabase → PostgreSQL puro

Módulo	Responsabilidade
<code>bridge/sql_transpiler.py</code>	Transpilação de dialeto SQL
<code>frontend_liberator.py</code>	Separação de camadas e <code>.env.production</code> limpo
<code>devops.py</code>	<code>DevOpsPackager</code> — Dockerfile não-root, Nginx OWASP, compose
<code>vsa_exporter.py</code>	Exportação do Quarteto e conector VSA
<code>auth_migrator.py</code>	Migração de contas preservando hash de senha
<code>jwt_generator.py</code>	Geração de segredos JWT para o ambiente self-hosted
<code>cloudflare_dns.py</code>	Criação e remoção de registros DNS
<code>unifier.py</code>	Fusão de múltiplas aplicações em fatias segregadas
<code>teardown.py</code>	Remoção atômica de stack, volumes, DNS e diretório
<code>pipeline_bridge.py</code>	<code>BridgePipeline</code> — orquestração das seis fases

Os sete invariantes do `AGENTS.md` são: libertação de fornecedor; ponte de dados com PostgREST; fusão multi-aplicação em fatias; migração real de autenticação com preservação de hash; teardown atômico com confirmação obrigatória; Quarteto Sine Qua Non exportado; e subordinação aos quatro portões dedicados.

15.5 Como funciona individualmente

Passo a passo. O pipeline completo (`unpack`) executa as seis fases descritas no capítulo 9. Os subcomandos individuais permitem operar fase a fase:

```
python ecossistema.py bridge scan --dir ../export-lovable
python ecossistema.py bridge convert-db --dir ../export-lovable
python ecossistema.py bridge merge --apps app1,app2
python ecossistema.py bridge pack --dominio meuapp.com
python ecossistema.py bridge migrate-auth --origem <postgres-origem> # preview
python ecossistema.py bridge migrate-auth --origem <postgres-origem> --apply # escreve
python ecossistema.py bridge destroy --stack meuapp # pede confirmação
python ecossistema.py bridge unpack ../export-lovable # pipeline completo
```

Portões. `G_BRIDGE_VENDOR_LOCKIN` (nenhum resquício proprietário), `G_BRIDGE_DOCKER_OCI` (boas práticas de imagem), `G_BRIDGE_POSTGRESQL` (SQL convertido válido) e `G_BRIDGE_VSA_COMPAT` (compatibilidade com o fatiamento do master).

Habilidades. `aidd-bridge` e `aidd-bridge-runner` são as skills do comando.

Determinismo. Integral — nenhuma das seis fases chama modelo.

Ferramentas acessadas. Docker e Docker Swarm, PostgreSQL, PostgREST, Nginx, API da Cloudflare (DNS), git.

Hooks e regras. Preview por padrão em `migrate-auth`; confirmação obrigatória em `destroy` salvo `--yes` explícito; UTF-8 forçado no subprocesso pelo `ecossistema.py`, uma decisão tomada especificamente porque o bridge imprime caracteres fora do cp1252 e derrubava o pipeline no console padrão do Windows.

Entrega. Entrega o projeto libertado com Docker, compose, banco convertido, `.env` limpo e Quarteto exportado. Entrega **para o aidd-master** dentro do fluxo e **para a VPS do usuário** ao final.

15.6 Como funciona dentro da camada FLUXO

Passo a passo. Etapa 3 do Fluxo 03. O orquestrador chama `bridge scan --dir <origem>`; o pipeline completo é acionado via `bridge unpack`.

Portões. Os quatro dedicados, mais o handoff Engine → Master, cujo campo `artefatos_frontend.origem_design` recebe `lovable_preserved` neste fluxo — registro explícito de que a interface original foi mantida.

Habilidades. `aidd-bridge`, `fluxo-03-runner`.

Determinismo. Integral — é o fluxo mais barato dos três.

Ferramentas acessadas. O export da plataforma low-code como entrada real.

Hooks e regras. Persistência `postgresql` no `arquitetura_alvo` do handoff (os Fluxos 01 e 02 usam `sqlite_wal`).

Entrega. Entrega ao `aidd-master` o projeto convertido com o Quarteto pronto para harmonização.

15.7 Como funciona dentro da camada ECOSSISTEMA

Passo a passo. Fora do fluxo, o bridge é usado cirurgicamente: converter só o banco, só empacotar, só migrar contas, só destruir uma stack.

Portões. `G_HADOLINT` e `G_INFRA_COMPOSE` do ecossistema auditam os artefatos Docker que ele produz.

Habilidades. `adaptar-lovable-aidd` cobre o caso de adaptação de projeto exportado do Lovable para projeto convencional.

Determinismo. Integral.

Ferramentas acessadas. Cloudflare, Docker Swarm, PostgreSQL.

Hooks e regras. Toda operação destrutiva exige confirmação; a Lei #6 aplicada ao projeto do usuário.

Entrega. Entrega ao ecossistema uma porta de entrada para projetos que nasceram fora dele — é o caminho pelo qual um sistema criado em plataforma proprietária passa a ser governado pelas mesmas leis dos sistemas nascidos aqui.

15.8 Rastreabilidade

`tools/aidd-bridge/AGENTS.md`; `tools/aidd-bridge/aidd_bridge/pipeline_bridge.py`; `tools/aidd-bridge/aidd_bridge/` (13 módulos); `tools/aidd-bridge/gates/` (4 portões); `ecossistema.py::cmd_bridge`; `scripts/orquestrador_sincrono.py::etapa_03_engine`.

Capítulo 16 — `aidd-master`: o harmonizador modular

Papel	Plataforma de arquitetura limpa modular em fatias verticais
Etapas na esteira	4 de 7 — primeira etapa do funil de convergência
Slash command	<code>/master <modulo></code>
CLI	<code>python ecossistema.py master add-module init compose audit ...</code> (22 subcomandos)
Portões locais	10, orquestrados por <code>`scripts/run_all.py`</code> com auto-remediação
Persistência	SQLite em modo WAL, com EventBus e mônada <code>`Result`</code>

16.1 O que é a ferramenta

`aidd-master` é onde os três fluxos convergem. Qualquer que seja o motor da etapa 3, o resultado passa por aqui e sai no mesmo formato: **monólito modular em fatias verticais** — domínio fatiado verticalmente, com uma camada horizontal compartilhada.

A analogia do `README.md` é a dos blocos de encaixe: permite acrescentar função nova sem quebrar o que já existe. A razão técnica é o isolamento de contexto delimitado.

16.2 O papel da ferramenta dentro do FLUXO

É a etapa 4 e a primeira do funil de convergência. O orquestrador roda `master init <slug>` e depois `master add-module <slug>`, criando a fatia vertical principal a partir do que o motor produziu.

A etapa existe porque os três motores produzem coisas estruturalmente diferentes — código autoral, integração de serviços, projeto low-code convertido — e o ecossistema precisa de uma forma canônica única para as etapas 5 e 6 operarem.

16.3 O papel da ferramenta dentro do ECOSSISTEMA

No ecossistema, o `aidd-master` é o **guardião da arquitetura alvo**. Os seus dez portões locais definem, na prática, o que "arquitetura correta" significa no AIDD: contexto delimitado isolado, fatia vertical completa, mônada `Result`, SQLite WAL, consulta parametrizada, exclusão lógica, observabilidade com SLA.

É também a fonte da linhagem do núcleo compartilhado: os arquivos que o `aidd-master` e o `aidd-enterprise` compartilham são auditados contra baseline por `G_DRIFT_NUCLEO_COMPARTILHADO`.

16.4 Como foi pensada, está estruturada e configurada

Os sete invariantes

- Isolamento de contexto delimitado e fatias verticais** (`G_ARQUITETURA`): import direto entre módulos de negócio é proibido; toda fatia contém `router.py`, `service.py`, `repository.py`, `dtos.py` e `events.py`; o frontend espelha com `hooks/`, `components/`, `types.ts` e `page.tsx`.
- Repositórios dedicados e isolamento de dados**: `JOIN` SQL entre contextos e chave estrangeira rígida entre fatias são proibidos; acesso a dado é encapsulado no `repository.py` da fatia; comunicação entre fatias só por `EventBus` ou interface pública de serviço.

3. **Mônada `Result`** (`G_QUALIDADE`): todo método de serviço retorna `Result[T, E]`; exceção crua nunca vaza para a apresentação.
4. **Persistência segura** (`G_SEGURANCA`): `PRAGMA journal_mode=WAL` sempre; marcador `?` em toda consulta; zero concatenação de string; exclusão lógica.
5. **Zero Stubs.**
6. **Observabilidade** (`G_PERFORMANCE`): rotinas decoradas com `@trace_span(name)` e teto de SLA `p99 < 200 ms`.
7. **Alinhamento com skills de engenharia**: fatias decompostas por `/aidd-tickets` e implementadas sob `/aidd-tdd` antes de graduar.

Os dez portões e a auto-remediação

`scripts/run_all.py` executa os dez portões em ordem fixa — `G_ESTRUTURA`, `G_SEGREDOS`, `G_SEGURANCA`, `G_TESTES`, `G_QUALIDADE`, `G_CONTRACTS`, `G_CHAOS`, `G_HARNESS_COMPAT`, `G_ARQUITETURA`, `G_INJECT` — com um comportamento distintivo: **se um portão falha, o orquestrador executa `scripts/autofix.py` e tenta de novo**; se ainda falhar, sai com 1. É auto-remediação determinística, não tolerância.

`G_CHAOS` merece destaque: testa resiliência por injeção de falha, e é o portão que distingue "passa nos testes" de "sobrevive a problema real".

Os 22 subcomandos

A CLI do master é a mais rica do ecossistema. Os principais grupos:

Grupo	Subcomandos
Ciclo de vida	<code>init</code> , <code>setup</code> , <code>status</code> , <code>apply</code> , <code>deploy</code>
Módulos	<code>add-module</code> , <code>refine-module</code> , <code>attach-vsa</code> , <code>heal</code>
Composição	<code>compose</code> , <code>compose-orca</code>
Qualidade	<code>audit</code> , <code>test</code> , <code>bench</code>
Frontend	<code>export-frontend</code> (Next.js/TypeScript tipado a partir do OpenAPI)
Infraestrutura	<code>scaffold-infra</code> (Terraform + Helm em <code>infra/</code>)
Componentes	<code>inject</code>

Dois merecem nota. `compose-orca` compõe módulos via subagentes efêmeros com purga de contexto — a aplicação direta da doutrina de isolamento cognitivo. `attach-vsa` conecta um backend VSA (núcleo, módulo e Quarteto) a um frontend preservado — por exemplo, a saída do `aidd-bridge` — **sem sobrescrever** Docker, Caddy ou o frontend existente; é a peça que torna o Fluxo 03 possível sem perder a interface original.

16.5 Como funciona individualmente

Passo a passo. `master init <slug> --pasta <p>` provisiona o projeto modular; `master add-module <nome>` gera a fatia vertical desacoplada com backend e frontend espelhados; `master export-frontend` deriva o frontend tipado do contrato OpenAPI; `master audit` roda os dez portões com auto-remediação; `master bench` mede concorrência real no SQLite WAL e no EventBus.

Portões. Os dez de `run_all.py`, mais `G_AST_BOUNDED_CONTEXT` para a auditoria AST de fronteira de contexto.

Habilidades. `aidd-master-runner` é a dona de `/master`; `/aidd-tickets` e `/aidd-tdd` governam a implementação de cada fatia.

Determinismo. Integral. Todo o scaffold, a derivação de frontend a partir do OpenAPI e a auditoria são mecânicos.

Ferramentas acessadas. SQLite (WAL), Alembic (migrações — a ferramenta tem `alembic/` e `alembic_models.py`), `behave` (suítes BDD em `refine-module`), `pytest`, Terraform e Helm (`scaffold-infra`), OpenAPI → TypeScript (`openapi_to_ts.py`).

Hooks e regras. Os sete invariantes; `CAPABILITIES.json` assinado com Ed25519 (`CAPABILITIES.json.ed25519.sig`) declara o que a ferramenta é capaz de fazer, de forma verificável.

Entrega. Entrega um projeto modular com fatias verticais completas, frontend tipado, banco WAL e contratos OpenAPI. Entrega **para o aidd-enterprise** dentro do fluxo.

16.6 Como funciona dentro da camada FLUXO

Passo a passo. Etapa 4. `master init <slug>` seguido de `master add-module <slug>`.

Portões. Os dez locais, mais a validação do handoff `handoff-master-to-enterprise.schema.json`, cujos campos obrigatórios são `versao_schema`, `diretorio_projeto`, `servidor_fastapi_ok`, `quarteto_sine_qua_non_rotas` e `componentes_para_blindagem`.

Habilidades. `componentes-runner`, `aidd-master-runner`, e os runners de fluxo.

Determinismo. Integral — etapa de custo zero em tokens.

Ferramentas acessadas. O resultado do motor da etapa 3, sob qualquer uma das três formas.

Hooks e regras. É aqui que a Lei #11 (Padrão-Ouro) se materializa no frontend Next.js e a Lei #10 (Quarteto) tem as rotas verificadas para o handoff.

Entrega. Entrega ao `aidd-enterprise` a lista de componentes para blindagem e a confirmação de que o servidor sobe e as rotas do Quarteto respondem.

16.7 Como funciona dentro da camada ECOSSISTEMA

Passo a passo. Fora do fluxo, `/master <modulo>` é o comando do dia a dia: adicionar uma fatia nova a um sistema em produção.

Portões. `G_ISOLATION_AUDIT`, `G_FRONTEND_LAYERS`, `G_ARQUITETURA_DELIVERABLE` e `G_DRIFT_ANALYZER` do ecossistema complementam os dez locais.

Habilidades. `/aidd-grill` antes de projetar a fatia; `/aidd-tdd` durante; `/aidd-diagnose` quando algo quebra.

Determinismo. Integral.

Ferramentas acessadas. `componentes/aidd-master/` guarda os componentes específicos da ferramenta no cofre canônico.

Hooks e regras. `G_DRIFT_NUCLEO_COMPARTILHADO` audita a linhagem compartilhada com o `aidd-enterprise` contra `gates/baseline_nucleo_compartilhado.json`.

Entrega. Entrega ao ecossistema a **definição executável de arquitetura correta** — os seus portões são a especificação operacional do que o AIDD considera um sistema bem construído.

16.8 Rastreabilidade

`tools/aidd-master/AGENTS.md`; `tools/aidd-master/scripts/aidd.py` (22 subcomandos); `tools/aidd-master/scripts/run_all.py`; `tools/aidd-master/scripts/gates/` (12 portões); `tools/aidd-master/CAPABILITIES.json` e a assinatura Ed25519; `componentes/compartilhado/specs/handoff-master-to-enterprise.schema.json`.

Capítulo 17 — `aidd-enterprise`: a blindagem criptográfica

Papel	Plataforma de missão crítica com injeção de componentes validada por SHA-256
Etapas na esteira	5 de 7
Slash command	<code>/enterprise <tipo> <nome></code>
CLI	<code>python ecossistema.py enterprise inject <tipo> <nome>, verificar-drift</code>
Portões locais	10, mesma bateria do master com <code>`G_INJECT`</code> estendido
Garantia central	Integridade criptográfica e detecção de drift pós-injeção

17.1 O que é a ferramenta

`aidd-enterprise` é o `aidd-master` com uma camada adicional de garantia: todo componente injetado é **validado contra assinatura SHA-256 antes de ser admitido em execução**, e a sincronização multi-harness é verificada por detecção de drift depois da injeção.

A distinção entre as duas ferramentas é de regime de confiança, não de arquitetura. O master constrói; o enterprise constrói **sob suspeita permanente** — zero-trust aplicado à própria cadeia de componentes.

17.2 O papel da ferramenta dentro do FLUXO

É a etapa 5. O orquestrador executa duas operações: `enterprise inject rule regra-integridade-<slug>` e `enterprise verificar-drift`. Ambas precisam sair com código 0.

O que essa etapa acrescenta ao produto é a garantia de que os componentes de governança presentes no projeto são exatamente os que deveriam estar ali — nem adulterados, nem dessincronizados entre assistentes.

17.3 O papel da ferramenta dentro do ECOSSISTEMA

No ecossistema, o enterprise é o **par de linhagem do master**. As duas ferramentas compartilham arquivos de núcleo, e essa duplicação é deliberada e auditada: o portão `G_DRIFT_NUCLEO_COMPARTILHADO` mantém um baseline em `gates/baseline_nucleo_compartilhado.json` e reprova divergência não documentada entre as duas linhagens.

É também a ferramenta que responde pela conformidade regulada: projetos que precisam demonstrar integridade de cadeia a um auditor externo passam por aqui.

17.4 Como foi pensada, está estruturada e configurada

Os seis invariantes do `AGENTS.md`: integridade criptográfica (validação SHA-256 antes da execução); mônada `Result` com exceção bloqueada no perímetro; contextos delimitados estritos com integração exclusivamente por `EventBus` e `core.*`; persistência resiliente (SQLite WAL, parametrização estrita, exclusão lógica); Zero Stubs com tipagem forte e testes automatizados; e ciclo `/aidd-tdd` com cobertura real de asserção **antes** de o componente receber a assinatura criptográfica.

O último invariante é o mais revelador: a assinatura não certifica que o código existe — certifica que ele passou por um ciclo TDD com asserções reais. Assinar código não testado seria transformar a criptografia em teatro.

O portão `G_INJECT` local valida duas dimensões: a infraestrutura do motor (contrato JSON Schema, arquivos de núcleo `profiles_registry.py`, `detector_camada.py`, `materializador.py`, `sincronizador_harness.py`, integração com a CLI e o `IntentRouter`, varredura AST anti-stub e suíte pytest dedicada) e a sincronização multi-harness com integridade SHA-256 pós-injeção via `sincronizador_harness.verificar_sincronizacao()`.

17.5 Como funciona individualmente

Passo a passo. `enterprise inject <tipo> <nome> --dir <projeto>` resolve o perfil do componente, materializa com transação, sincroniza para os assistentes, calcula os hashes e verifica a integridade. `enterprise verificar-drift --dir <projeto>` recalcula e compara contra o registro. `python scripts/run_all.py` roda os dez portões locais.

Portões. Os dez de `run_all.py`, com `G_INJECT` estendido para a verificação criptográfica.

Habilidades. `aidd-enterprise-runner` é a dona de `/enterprise`; `aidd-master-pack` cobre a suíte cross-domain.

Determinismo. Integral.

Ferramentas acessadas. SHA-256 e Ed25519 (`assinatura_manifesto.py` do núcleo), Alembic, SQLite WAL, pytest.

Hooks e regras. Zero-trust: nenhum componente entra em execução sem validação de assinatura.

Entrega. Entrega componentes blindados e um veredito de drift. Entrega **para o** `aidd-ops` dentro do fluxo.

17.6 Como funciona dentro da camada FLUXO

Passo a passo. Etapa 5: `inject` seguido de `verificar-drift`.

Portões. Os dez locais, mais a validação do handoff `handoff-enterprise-to-ops.schema.json`, cujos campos obrigatórios são `versao_schema`, `diretorio_projeto`, `sha256_audit_ok`, `drift_verificado` e `manifesto_deploy`.

Habilidades. `aidd-enterprise-runner` e os runners de fluxo.

Determinismo. Integral — etapa de custo zero em tokens.

Ferramentas acessadas. O projeto harmonizado pelo master.

Hooks e regras. O manifesto de deploy declara `tipo_runtime`, presença de Dockerfile e compose, e as portas expostas (80, 443, 3000).

Entrega. Entrega ao `aidd-ops` a confirmação de integridade e o manifesto de deploy.

17.7 Como funciona dentro da camada ECOSSISTEMA

Passo a passo. Fora do fluxo, `/enterprise inject` é o caminho para introduzir componente certificado em qualquer projeto AIDD.

Portões. `G_DRIFT_NUCLEO_COMPARTILHADO`, `G_SUPPLY_CHAIN` e `G_COMPONENTE_AGNOSTICO` do ecossistema.

Habilidades. `aidd-componentes`, `aidd-dependencias`.

Determinismo. Integral.

Ferramentas acessadas. `componentes/compartilhado/src-core/package_verifier.py` e `assinatura_manifesto.py`.

Hooks e regras. A regra de que assinatura só vem depois de TDD com asserção real.

CONFORMIDADE DA HONESTIDADE DE RÓTULO

`G_HONESTIDADE_ROTULO` foi plenamente reabilitado e roda como portão obrigatório em todo commit (`always_run: true`). As mensagens de saída de todos os scripts de portão em `gates/`, `tools/aidd-master` e `tools/aidd-enterprise` foram alinhadas à linguagem técnica factual, auditando com zero termos proibidos.

Entrega. Entrega ao ecossistema a garantia de que a cadeia de componentes é verificável — e o registro honesto de onde ela ainda não está.

17.8 Rastreabilidade

`tools/aidd-enterprise/AGENTS.md`; `tools/aidd-enterprise/scripts/aidd.py`; `tools/aidd-enterprise/scripts/run_all.py`; `tools/aidd-enterprise/scripts/gates/G_INJECT.py`; `tools/aidd-enterprise/scripts/injector/`; `componentes/compartilhado/src-core/assinatura_manifesto.py`; `gates/G_DRIFT_NUCLEO_COMPARTILHADO.py` e `gates/baseline_nucleo_compartilhado.json`.

Capítulo 18 — `aidd-ops`: a infraestrutura agêntica

Papel	Meta-orquestrador de infraestrutura para stacks self-hosted
Etapa na esteira	6 de 7 — última etapa antes da auditoria
Slash command	<code>/ops [requisito]</code>
CLI	<code>python ecossistema.py ops plan bootstrap preflight deploy</code>
Pipeline próprio	3 fases: Intake → Curadoria → Sizing
Artefato central	<code>PLANO-INFRAESTRUTURA.json</code>

18.1 O que é a ferramenta

`aidd-ops` transforma um requisito em linguagem natural — "uma clínica com agendamento e prontuário para 2 mil pacientes" — em uma infraestrutura dimensionada, provisionada e monitorada: sizing de VPS, hardening de SSH, Docker, Traefik, PostgreSQL, Uptime Kuma e bateria de preflight de ponta a ponta.

18.2 O papel da ferramenta dentro do FLUXO

É a etapa 6. No orquestrador síncrono atual, a etapa se limita a **validar** a presença de `Dockerfile` e `docker-compose.yml` no projeto — o provisionamento real da VPS é acionado separadamente, porque envolve credencial, host remoto e custo. Esse desacoplamento é deliberado: a Lei #7 (Desenvolvedor no Controle) não permite que um fluxo automatizado provisione infraestrutura paga sem decisão humana explícita.

18.3 O papel da ferramenta dentro do ECOSSISTEMA

No ecossistema, o `aidd-ops` é a **fonte do plano de infraestrutura** — o mesmo `PLANO-INFRAESTRUTURA.json` que a `aidd-factory` consome e que o `aidd-planner` exporta. As três ferramentas compartilham um único esquema, e o caminho do nicho dinâmico descrito nos capítulos 8 e 12 nasceu aqui.

É também a ferramenta que aplica a política anti-NIH de forma mais visível: em vez de escrever monitoramento, usa os templates oficiais do Uptime Kuma; em vez de escrever scripts de hardening, usa a coleção Ansible `devsec.hardening`.

18.4 Como foi pensada, está estruturada e configurada

As três fases do pipeline

Fase	Módulo	O que faz
1	<code>01_intake.py</code>	Reconhece o nicho a partir do texto; suporta nicho dinâmico com prefixo <code>dinamico_</code>
2	<code>02_curadoria.py</code>	Cura a stack de ferramentas para o nicho reconhecido
3	<code>03_sizing.py</code>	Dimensiona recursos — e é 100% orientado a nome de ferramenta

`pipeline_ops.py` roda as três em sequência, grava `PLANO-INFRAESTRUTURA.json` no destino acumulando o estado de cada fase e imprime um resumo legível. Propaga códigos de erro estruturados das fases — não mascara `Result.fail()`.

O `AGENTS.md` registra um detalhe importante sobre a Fase 3: ela **não precisou de alteração** para suportar nicho dinâmico, porque já era inteiramente orientada a nome de ferramenta e ignora graciosamente ferramentas desconhecidas. É um bom exemplo de desenho que envelhece bem.

Os seus invariantes

Compose determinístico e estruturado sob `templates/infra/ (G_INFRA_COMPOSE)`; boas práticas OCI com Hadolint e zero violação alta ou crítica (`G_HADOLINT`); observabilidade ativa **apenas** com templates oficiais do Uptime Kuma, zero painel falso ou simulado (anti-NIH #14); hardening idempotente via coleção Ansible `devsec.hardening`, zero script shell improvisado (anti-NIH #15); Zero Stubs; e triagem obrigatória por `/aidd-diagnose` antes de qualquer correção emergencial em incidente de produção.

O núcleo operacional

Módulo de <code>src/core/</code>	Responsabilidade
<code>ssh_runner.py</code>	Execução remota via SSH
<code>cofre_credenciais.py</code>	Cofre local com <code>sops</code> + <code>age</code>
<code>preflight.py</code>	Bateria E2E pós-deploy
<code>compose_preflight.py</code>	Validação de compose antes do deploy
<code>uptime_kuma.py</code>	Integração com a API do Uptime Kuma
<code>coolify.py</code>	Integração com Coolify
<code>result.py</code>	Mônada <code>Result</code> local

A DECISÃO DO COFRE: SOPS + AGE, E NÃO VAULTWARDEN

O cabeçalho de `cofre_credenciais.py` registra a decisão e o motivo. O Vaultwarden foi descartado por trazer serviço e banco de dados para um caso de uso que é **pipeline automatizado** — decifrar sob demanda antes de `docker compose up` — e não humano compartilhando senha por interface gráfica.

A escolha foi `sops` (Mozilla), que cifra o **valor** de cada variável de um arquivo dotenv mantendo as **chaves** legíveis — o que preserva a capacidade de comparar versões — usando `age` (FiloSottile) como backend assimétrico moderno (X25519 + ChaCha20-Poly1305). Sem servidor, sem banco, sem processo daemon.

A bateria de preflight

`preflight.py` é uma bateria determinística contra um deploy já realizado, com quatro verificações: healthcheck HTTP (`/healthz` de cada serviço, status 200 obrigatório); validação de certificado SSL/TLS (emissor, vigência, cadeia); resolução DNS dos subdomínios da topologia, com resolver injetável; e simulação de um webhook de ponta a ponta com disparo sintético no gateway. Suporta injeção de dependência para execução hermética em teste unitário e devolve `Result` com relatório estruturado em JSON.

18.5 Como funciona individualmente

Passo a passo.

```
python ecossistema.py ops plan "clinica com agendamento e prontuario" --pasta ../infra
python ecossistema.py ops plan --ferramentas-json ferramentas.json --pasta ../infra # nicho dinâmico
python ecossistema.py ops bootstrap --host <ip> # hardening + Docker via SSH
python ecossistema.py ops deploy # deploy E2E com Result e rollback
python ecossistema.py ops preflight # bateria E2E pós-deploy
python scripts/pipeline_ops.py monitor export|check # sondas do Uptime Kuma
```

Portões. `G_OPS_MVP` e `G_OPS_SSH` locais; `G_INFRA_COMPOSE` e `G_HADOLINT` globais.

Habilidades. `aidd-ops-runner` é a dona de `/ops`; `/aidd-diagnose` é obrigatória antes de correção emergencial.

Determinismo. Integral nas três fases do pipeline.

Ferramentas acessadas. Docker e Docker Compose, Traefik, PostgreSQL, Authentik, Chatwoot, Cal.com, Twenty (templates em `templates/infra/`), Uptime Kuma, Ansible (`devsec.hardening`), `sops` + `age`, Helm (`charts/aidd-ops/`), SSH, Coolify, Hadolint, Checkov.

Hooks e regras. Idempotência em todo hardening; zero script improvisado; templates oficiais apenas.

Entrega. Entrega `PLANO-INFRAESTRUTURA.json`, manifestos compose, VPS provisionada e relatório de preflight em JSON. Entrega **para a aidd-factory** (o plano) e **para a produção** (a infraestrutura).

18.6 Como funciona dentro da camada FLUXO

Passo a passo. Etapa 6: valida a presença dos manifestos Docker no projeto. O provisionamento real fica fora do fluxo automatizado, por decisão de governança.

Portões. `G_INFRA_COMPOSE` e `G_HADOLINT` sobre os manifestos.

Habilidades. `aidd-ops-runner`, runners de fluxo.

Determinismo. Integral — etapa de custo zero em tokens.

Ferramentas acessadas. O manifesto de deploy do handoff Enterprise → Ops.

Hooks e regras. Lei #7: infraestrutura paga exige decisão humana explícita.

Entrega. Entrega à etapa 7 (auditoria) a confirmação de que os manifestos existem e são válidos.

18.7 Como funciona dentro da camada ECOSSISTEMA

Passo a passo. Fora do fluxo, o `aidd-ops` é a ferramenta de operação contínua: planejar, provisionar, implantar, monitorar, rotacionar segredo (`scripts/rotate_secrets.py`).

Portões. Os globais de infraestrutura.

Habilidades. `/aidd-diagnose` para triagem científica de incidente em cinco fases, integrada ao grafo de conhecimento.

Determinismo. Integral.

Ferramentas acessadas. `data/catalogo_nichos.json` (cinco nichos fixos), `data/requisitos_recursos.json` (tabela de dimensionamento).

Hooks e regras. As duas regras anti-NIH (#14 Uptime Kuma oficial, #15 Ansible `devsec.hardening`) são o traço mais característico da ferramenta: ela prefere consistentemente integrar o que já existe a escrever o próprio.

Entrega. Entrega ao ecossistema o contrato de infraestrutura compartilhado e, ao usuário, um sistema em produção com observabilidade real.

18.8 Rastreabilidade

`tools/aidd-ops/AGENTS.md`; `tools/aidd-ops/scripts/pipeline_ops.py`; `tools/aidd-ops/scripts/phases/` (3 fases); `tools/aidd-ops/src/core/` (7 módulos); `tools/aidd-ops/templates/infra/`; `tools/aidd-ops/ansible/playbooks/hardening.yml`; `tools/aidd-ops/gates/`; `tools/aidd-ops/data/catalogo_nichos.json`.

PARTE IV — CAMADAS TRANSVERSAIS

As três partes anteriores percorreram o ecossistema verticalmente: do organismo aos fluxos, dos fluxos às ferramentas. Esta parte corta na horizontal. Os quatro capítulos seguintes tratam de camadas que atravessam **todas** as ferramentas e **todos** os fluxos: as habilidades e comandos, o catálogo completo de portões, a malha de agnosticidade e a síntese consolidada da economia de tokens.

Capítulo 19 — Habilidades e comandos: a interface universal

19.1 O que é uma habilidade no AIDD

Uma habilidade (skill) é um pacote de instruções versionado que qualquer assistente de IA carrega para executar um procedimento específico do ecossistema. No AIDD, uma habilidade nasce em `componentes/compartilhado/skills/<nome>/` — a fonte física canônica — e é distribuída para os dez ambientes de assistente por `python ecossistema.py components sync`.

O repositório tem **60 habilidades** nessa pasta e **14 comandos** em `componentes/compartilhado/comandos/`.

19.2 As famílias de habilidades

Runners de ferramenta

Um por ferramenta, cada um dono de exatamente um slash command:

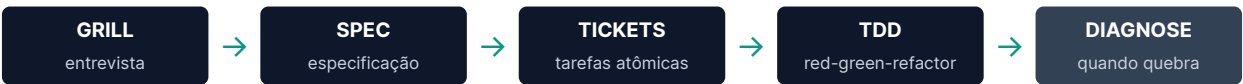
Habilidade	Comando	Ferramenta
<code>aidd-forge-runner</code>	<code>/forge</code>	<code>aidd-forge</code>
<code>aidd-planner-runner</code>	<code>/planner</code>	<code>aidd-planner</code>
<code>aidd-generator-runner</code>	<code>/generate</code>	<code>aidd-generator</code>
<code>aidd-factory-runner</code>	<code>/factory</code>	<code>aidd-factory</code>
<code>aidd-bridge-runner</code>	<code>/bridge</code>	<code>aidd-bridge</code>
<code>aidd-master-runner</code>	<code>/master</code>	<code>aidd-master</code>
<code>aidd-enterprise-runner</code>	<code>/enterprise</code>	<code>aidd-enterprise</code>
<code>aidd-ops-runner</code>	<code>/ops</code>	<code>aidd-ops</code>

Runners de fluxo

`aidd-pure` e `fluxo-01-runner`; `aidd-open` e `fluxo-02-runner`; `aidd-freedom` e `fluxo-03-runner` (com operações atômicas da engine via `aidd-bridge-runner`). O par existe porque uma habilidade é a dona do slash command e a outra é o motor que executa a esteira.

Habilidades procedimentais de engenharia

As sete habilidades anti-vibe coding descritas no capítulo 2 — `/aidd-grill`, `/aidd-grill-docs`, `/aidd-spec`, `/aidd-tickets`, `/aidd-tdd`, `/aidd-diagnose` e `/aidd-handoff` — formam uma cadeia natural:



Habilidades de evolução e orquestração

`melhoria`, `plan`, `orchestrate` (as três donas do fluxo de evolução); `planos-auditoria-runner` e `orca-plan-orchestrator` (os motores); `aidd-orca`, `aidd-orchestrate`, `aidd-orchestrator-runner`, `aidd-planos`, `aidd-melhoria`.

Habilidades de gestão do próprio ecossistema

`aidd-componentes` e `componentes-runner` (componentes); `aidd-dependencias` e `dependencia-runner` (dependências de terceiros); `aidd-skills` e `skill-creator-runner` (criação de habilidades); `aidd-mcp` e `mcp-creator-runner` (servidores MCP); `explore-codebase`, `refactor-safely`, `review-changes`, `debug-issue`.

Habilidades de domínio externo

O ecossistema integra habilidades de terceiros como dependências declaradas: `cloudflare`, `cloudflare-one`, `cloudflare-email-service`, `cloudflare-one-migrations`, `durable-objects`, `nextjs-on-cloudflare`, `workers-best-practices`, `wrangler`, `turnstile-spin`, `web-perf`, `sandbox-next`, `sandbox-stable`, `sandbox-migrate-to-next`, `agents-sdk`, `impeccable`, `sandeco-token-reduce`.

19.3 A regra: um comando, um dono

Já enunciada no capítulo 2, ela merece repetição aqui porque é a regra que estrutura toda a camada de habilidades: **cada slash command tem exatamente uma habilidade dona**; habilidades-motor não têm comando próprio e são acionadas pela dona.

A consequência prática é que, ao procurar quem responde por um comando, existe sempre uma resposta única. A consequência histórica é que a regra nasceu de um incidente: duas habilidades respondendo ao mesmo comando com regras opostas causaram mesas recursivas dentro de mesas no aplicativo de orquestração em 11 de setembro de 2026.

19.4 A distribuição física

O CAMINHO DE UM COMPONENTE, DO NASCIMENTO À DISTRIBUIÇÃO

1. Nasce em `componentes/compartilhado/skills/<nome>/SKILL.md` (fonte canônica).
2. `python ecosistema.py components sync --tipo skills` distribui.
3. Chega em `.claude/skills/`, `.opencode/skills/`, `.mimocode/skills/`, `.agents/skills/`, `.gemini/extensions/`, `.cursor/rules/` e demais destinos.
4. `G_COMPONENTE_AGNOSTICO` audita a cobertura contra o manifesto.
5. `G_UNIVERSAL_HARNESS` audita o **wiring** real em cada assistente.
6. `G_HARNESS_COMPAT` audita a sincronia entre os artefatos da raiz.

Editar um destino diretamente inverte o fluxo e é o erro que os portões detectam: o destino é sobrescrito no próximo `sync`, e a alteração se perde.

Há ainda o modo de reparo: `components sync --force` restaura destinos divergentes e órfãos a partir da fonte, e `components verify` audita sem escrever.

19.5 Rastreabilidade

`componentes/compartilhado/skills/` (60 habilidades); `componentes/compartilhado/comandos/` (14 comandos); `scripts/gestor_componentes.py`; `ecosistema.py:cmd_components`; `docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md` §3 e §5.

Capítulo 20 — O catálogo completo de portões

20.1 A distribuição dos 145 portões

O repositório tem 145 arquivos `G_*.py`. Eles se distribuem em quatro camadas:

Camada	Onde	Quantidade	Papel
Portões globais	<code>gates/</code>	24	Auditam o ecossistema inteiro
Portões de ferramenta	<code>tools/<ferramenta>/gates/</code> ou <code>scripts/gates/</code>	~55	Auditam a ferramenta e o que ela produz
Portões-template	<code>tools/aidd-forge/aidd_forge/templates/gates/</code>	8	São injetados nos projetos gerados
Portões de projeto	Projetos gerados	variável	Cópias dos templates, ativas no projeto do usuário

A contagem de 145 inclui as cópias distribuídas — o número de portões **distintos** é menor, e a duplicação entre `aidd-master` e `aidd-enterprise` é justamente o que `G_DRIFT_NUCLEO_COMPARTILHADO` mantém sob controle.

20.2 Portões por ferramenta

Ferramenta	Portões locais
<code>aidd-forge</code>	<code>G_BLOQUEAR_SEGREDOS</code> , <code>G_CONTRACTS</code> , <code>G_CYBERSECURITY_OWASP</code> , <code>G_ESTRUTURA_AST</code> , <code>G_HARNESS_COMPAT</code> , <code>G_INJECT</code> , <code>G_PERFORMANCE</code> , <code>G_TESTES_REAIS</code>
<code>aidd-planner</code>	<code>G_PLANNER_SCHEMA</code> , <code>G_PLANNER_SINE_QUA_NON</code> , <code>G_PLANNER_COERENCIA_FLUXO</code>
<code>aidd-generator</code>	<code>G_TOKENOMICS</code> , <code>G_CYBERSECURITY_OWASP</code> , <code>G_SANDBOX_NIVEL_1</code> , <code>G_SESSAO_HERMETICA</code> , <code>G_BLOQUEAR_SEGREDOS</code> , <code>G_INTEGRACAO_CROSS_SCRIPT</code> , <code>G_VERIFICAR_LLM_PRONTO</code> , <code>G_INJECT</code> , <code>G_HARNESS_COMPAT</code> , <code>AUDITAR_COMPARATIVO_HARNESS</code>
<code>aidd-factory</code>	<code>G_FACTORY_INPUT</code> , <code>G_FACTORY_OUTPUT</code> , <code>G_FACTORY_DETERMINISTIC</code>
<code>aidd-bridge</code>	<code>G_BRIDGE_VENDOR_LOCKIN</code> , <code>G_BRIDGE_DOCKER_OCI</code> , <code>G_BRIDGE_POSTGRESQL</code> , <code>G_BRIDGE_VSA_COMPAT</code>
<code>aidd-master</code>	<code>G_ESTRUTURA</code> , <code>G_SEGREDOS</code> , <code>G_SEGURANCA</code> , <code>G_TESTES</code> , <code>G_QUALIDADE</code> , <code>G_CONTRACTS</code> , <code>G_CHAOS</code> , <code>G_HARNESS_COMPAT</code> , <code>G_ARQUITETURA</code> , <code>G_INJECT</code> , <code>G_AST_BOUNDED_CONTEXT</code> , <code>G_PERFORMANCE</code>
<code>aidd-enterprise</code>	A mesma bateria do master, com <code>G_INJECT</code> estendido para verificação criptográfica
<code>aidd-ops</code>	<code>G_OPS_MVP</code> , <code>G_OPS_SSH</code>

20.3 As categorias de auditoria

Os 39 portões globais podem ser lidos por intenção, e essa leitura revela a estratégia de qualidade do ecossistema:

Categoria	Portões
Integridade estrutural	<code>G_ECOSSISTEMA_INTEGRIDADE</code> , <code>G_ESCRITOR_ATOMICO</code> , <code>G_TRANSACTION_LOG_LRU</code>

Categoria	Portões
Arquitetura	G_ARQUITETURA_DELIVERABLE , G_ISOLATION_AUDIT , G_FRONTEND_LAYERS , G_DRIFT_ANALYZER , G_PROTOTYPE_REWRITE
Agnosticidade	G_HARNESS_COMPAT , G_UNIVERSAL_HARNESS , G_COMPONENTE_AGNOSTICO , G_PROTOCOL_FALLBACK
Segurança	G_SEGREDOS , G_LLM_PROMPT_SHIELD , G_SUPPLY_CHAIN , G_DEPENDENCIAS_PIN_HASH
Infraestrutura	G_INFRA_COMPOSE , G_HADOLINT
Honestidade	G_HONESTIDADE_ROTULO , G_TESTES_REAIS , G_CLI_HELP_CONSISTENCIA , G_DOCS_ROT , G_LIVRO_EVIDENCIA
Governança agêntica	G_ZERO_HEADLESS , G_ORQUESTRADOR_SINCRONO , G_DRIFT_NUCLEO_COMPARTILHADO
Meta-portões	G_PORTAO_PROVA_QUE_MORDE (Lei #13), G_LEI_DECLARA_PORTAO (Lei #8)
Anti-rot por Lei (fecham as Leis 1, 2, 3, 4, 9, 10, 11 — Sessões 12-24)	G_DETERMINISMO_LEI_1 , G_SAIDA_BINARIA , G_MIGRATION_ROT , G_ESTRUTURA_ESTADO , G_IDIOMA_LEI_4 , G_ENV_ROT , G_SKILL_ROT , G_DISCIPLINA_TESTE_FERRAMENTA , G_CONTRACT_ROT , G_QUARTETO_SINE_QUA_NON , G_STACK_PADRAO_OURO

Três portões merecem nota especial porque auditam coisas que a maioria dos projetos não audita. [G_CLI_HELP_CONSISTENCIA](#) compara, por AST, as flags citadas em `print()` e `raise()` contra as flags realmente definidas em `add_argument` nos pontos de entrada `argparse` das ferramentas. Ele impede que a mensagem de erro sugira uma flag que não existe — um defeito comum e invisível para teste funcional.

[G_TESTES_REAIS](#) roda `pytest` de verdade em cada `tools/<ferramenta>` e falha se qualquer suíte tiver `failed > 0`. É a Lei #8 tornada mecânica: não se pode alegar suíte verde sem suíte verde.

[G_LLM_PROMPT_SHIELD](#) audita, por AST, se os clientes de modelo de linguagem usam deterministicamente o `PromptShield` para sanitização contra injeção de prompt e jailbreak. É um portão de segurança específico de sistemas agênticos.

20.4 O ciclo de teste de ferramenta

A Lei #9 define um ciclo de cinco passos, documentado em [docs/protocolos/PROTOCOLO-TESTES-FERRAMENTAS.md](#), que precede qualquer declaração de conformidade: (1) corrigir bugs automaticamente até 100% de conformidade, com zero inconsistência; (2) commit e push; (3) limpar o projeto alvo; (4) executar de forma limpa; (5) atualizar o relatório em [docs/teste-end-to-end/](#).

O passo 3 é o que distingue esse ciclo de um teste comum: limpar o projeto alvo impede que a execução se apoie em artefato residual de uma execução anterior.

20.5 Rastreabilidade

[gates/](#) (39 portões e suítes); [tools/*/gates/](#) e [tools/*/scripts/gates/](#); `.pre-commit-config.yaml`; [docs/protocolos/PROTOCOLO-TESTES-FERRAMENTAS.md](#); [docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md](#) §4.

Capítulo 21 — Agnosticidade: escrever uma vez, rodar em todos

21.1 As quatro dimensões de agnosticidade

A Lei #6 — Supremacia Agnóstica — proíbe dependência de fornecedor em quatro dimensões simultâneas, e cada uma tem uma implementação concreta.

Sistema operacional. Tudo é Python puro. As poucas concessões a plataforma são explícitas e tratadas: `PYTHONIOENCODING=utf-8` forçado no subprocesso para o console do Windows, `sys.stdout.reconfigure(encoding='utf-8')` no topo dos portões, e invólucros `.sh` e `.cmd` para os hooks compartilhados.

Assistente. Dez ambientes sincronizados a partir de uma fonte única, com três portões auditando a paridade.

Modelo. O pipeline opera de duas formas: usando o modelo do próprio assistente da sessão, via protocolo delegado, sem custo adicional de API; ou via provedor externo local ou em nuvem (Ollama, Groq, Anthropic, OpenAI), configurado por `LLM_MODEL` e credencial. `preflight_llm.py` verifica os dois antes de começar.

Protocolo. `G_PROTOCOL_FALLBACK` garante que nada seja exposto exclusivamente por MCP sem contrapartida REST/OpenAPI. Um sistema cuja única porta fosse o MCP estaria preso ao ecossistema de agentes — o oposto do objetivo.

21.2 O protocolo delegado

O protocolo delegado é o mecanismo pelo qual o pipeline usa o modelo **do assistente que está conduzindo a sessão**, em vez de abrir uma conexão própria com um provedor. Quando o pipeline precisa de uma decisão de modelo, ele emite uma solicitação estruturada; o assistente responde; o pipeline valida a resposta contra o esquema e continua.

A consequência econômica é direta: quem já paga pela sessão do assistente não paga uma segunda vez em API. A consequência arquitetural é que o mesmo código roda com qualquer modelo, porque a fronteira é um contrato JSON, não um SDK.

Todo acesso a modelo no gerador passa por `solicitar_llm()` de `utils_delegacao.py` — o `AGENTS.md` da Fase 2 proíbe explicitamente chamada direta a `litellm`. Essa centralização é o que torna a troca de provedor uma mudança de configuração em vez de uma refatoração.

21.3 O manifesto de harness

`gates/manifesto_harnesses.json` (13 KB) é o registro canônico de qual componente deve existir em qual assistente, em qual caminho. `G_COMPONENTE_AGNOSTICO` audita todo componente novo ou modificado contra esse manifesto, e `scripts/harness_hygiene.py` mantém a higiene dos diretórios.

A proveniência do mapeamento está registrada: testes empíricos contra instalações reais dos sete assistentes nesta máquina, em 5 de setembro de 2026. Onde a confirmação não foi possível, o registro diz isso — o caso de `.gemini/skills/`, sincronizado mas com `confirmado: false`, e o do Freebuff, instalado mas sem modo não interativo para validação automatizada.

21.4 Rastreabilidade

```
AGENTS.md  Lei  #6; docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md  §5; docs/protocolos/05-09-2026_
protocolo-agnosticidade-componentes.md; gates/manifesto_harnesses.json; gates/G_UNIVERSAL_HARNESS.py;
gates/G_COMPONENTE_AGNOSTICO.py; gates/G_PROTOCOL_FALLBACK.py; tools/aidd-generator/scripts/phases/
utils_delegacao.py.
```


Capítulo 22 — Síntese: a economia de tokens ponta a ponta

Este capítulo consolida, numa visão única, o que os capítulos anteriores trataram dispersamente. É o eixo "economia de tokens" visto simultaneamente nos três níveis.

22.1 A matriz consolidada

Mecanismo	Nível	Onde vive	Efeito
Determinismo obrigatório	Macro	Lei #1; 145 portões; 5 das 7 etapas de fluxo	Elimina a chamada em vez de baratear
Protocolo Caveman tri-fase	Macro	src-core/caveman_protocol.py	30–50% de redução na entrada, em contagem BPE
Linter de Caveman	Micro	generator/scripts/core/caveman_linter.py	Garante a tríade ENTRADA/COT/SAÍDA por AST, sem gastar token
Fatiamento de contexto	Macro	core/context_slicer.py	Payload de assinaturas abaixo de 150 tokens
Grafo antes de busca textual	Macro	AGENTS.md §1 ; MCP code-review-graph	Evita leitura de arquivo inteiro e grep exploratório
Descarregamento de esquema MCP	Macro	core/mcp_dynamic_router.py	Remove milhares de tokens fixos do prompt de sistema
Estado em arquivo	Meso	Handoffs JSON; cognitive_ledger.py	Substitui histórico conversacional por ~5k tokens tipados
Retomada inteligente	Micro	generator/scripts/core/pipeline_state.py	--resume pula fase completa com artefato válido
Micro-ambiente por fase	Micro	generator/scripts/phases/phase_*/AGENTS.md	Só as regras da fase corrente entram em memória
Orçamento formal por fase	Micro	generator/config/token_budgets.json	Teto declarado + limiar de desvio de 1,2
Auditoria de tokenomics	Micro	G_TOKENOMICS	Reprova estouro de orçamento e rótulo de sonesto de medição
Benchmark real	Micro	generator/scripts/benchmark_tokenomics.py	Mede com tiktoken contra baseline legada
Purga de contexto de subagente	Macro	forge/core/subagent_purger.py	Subagente morre após validação AST
Disciplina de terminal	Macro	AGENTS.md §1	tail/grep obrigatórios; zero despejo de log ou lockfile
Edição por busca e substituição	Macro	AGENTS.md §1	Nunca reescrever arquivo inteiro na saída
Executor silencioso	Macro	AGENTS.md §1	Status de uma linha; zero repetição de código na conversa
Protocolo delegado	Macro	generator/scripts/phases/utils_delegacao.py	Usa o modelo da sessão; zero custo adicional de API

22.2 Onde o token é realmente gasto

Num Fluxo 01 completo com implementação de código, o consumo se concentra em quatro fases do gerador, com orçamento total declarado de 115 mil tokens. As outras seis etapas do fluxo — forge, master, enterprise, ops, auditoria e as quatro fases determinísticas do próprio gerador — custam zero.

Nos Fluxos 02 e 03, o consumo cai drasticamente: o Fluxo 02 usa modelo apenas em três das suas fases de integração, e o Fluxo 03 não usa modelo em nenhuma das seis fases do pipeline de libertação.

A CONCLUSÃO PRÁTICA

A pergunta "quanto custa gerar um sistema com o AIDD?" tem três respostas, e a escolha do fluxo é a decisão de maior impacto econômico do ecossistema. Escrever do zero custa dezenas de milhares de tokens; integrar open-source custa uma fração disso; libertar um projeto low-code custa praticamente nada em tokens — o custo ali é de tempo de máquina, não de modelo.

22.3 A honestidade da métrica

O ecossistema audita a própria alegação de economia. [G_TOKENOMICS](#) reprova quando uma fase afirma "medição real" para um valor autodeclarado, e exige que a origem da medição esteja rotulada como [autodeclarado](#) ou [medido_api](#). O [benchmark_tokenomics.py](#) produz medição real via [tiktoken](#), com custo em dólares e resultado de pytest no mesmo relatório — porque economia que degrada qualidade não é economia.

Este livro segue a mesma regra: os orçamentos citados são **valores declarados em [token_budgets.json](#)**, não medições de execução. Onde há medição real disponível, ela vive nos relatórios de benchmark do repositório, não aqui.

22.4 Rastreabilidade

Todos os arquivos citados na matriz da seção 22.1, mais [AGENTS.md](#) Lei #4 e §1.

Apêndice A — Referência da CLI unificada

Todos os comandos abaixo são despachados por `ecossistema.py` e existem também na forma de slash command, conforme a dupla camada descrita no capítulo 2.

A.1 Fluxos da Tríade Canônica

Comando	Alias	O que faz
<code>python ecossistema.py pure</code>	<code>aidd-pure</code>	Fluxo 01 — do zero puro, motor <code>aidd-generator</code>
<code>python ecossistema.py open</code>	<code>aidd-open</code>	Fluxo 02 — motores open-source, motor <code>aidd-factory</code>
<code>python ecossistema.py freedom</code>	<code>aidd-freedom</code>	Fluxo 03 — liberação de low-code, motor <code>aidd-bridge</code>
<code>python ecossistema.py run-fluxo --fluxo <pure\ open\ freedom></code>	—	Forma longa, aceita <code>--dry-run</code>

Parâmetros comuns aos fluxos: `--nome`, `--slug`, `--dominio`, `--pasta`, `--origem` (Fluxo 03), `--dry-run`.

A.2 Ferramentas

Comando	Ferramenta
<code>forge init [caminho] [--force]</code>	<code>aidd-forge</code>
<code>forge inject <tipo> <nome></code>	<code>aidd-forge</code>
<code>forge audit <caminho> [--fmt] [--output]</code>	<code>aidd-forge</code>
<code>forge conform <caminho> [--dry-run] [--items N]</code>	<code>aidd-forge</code>
<code>planner init --fluxo <1\ 2\ 3> --nome --slug --dominio --pasta</code>	<code>aidd-planner</code>
<code>planner validate <caminho></code>	<code>aidd-planner</code>
<code>planner export <caminho> --formato factory</code>	<code>aidd-planner</code>
<code>planner audit <pasta></code>	<code>aidd-planner</code>
<code>generate "<ideia>" --pasta <p> [--implementar-codigo] [--resume]</code>	<code>aidd-generator</code>
<code>factory --plano <arquivo> --pasta <destino></code>	<code>aidd-factory</code>
<code>factory curate --dominio <d> --output <pasta></code>	<code>aidd-factory</code>
<code>bridge scan\ convert-db\ merge\ pack\ migrate-auth\ destroy\ unpack</code>	<code>aidd-bridge</code>
<code>master init\ add-module\ compose\ compose-orca\ audit\ test\ bench\ heal\ status\ deploy\ apply\ setup\ inject\ attach-vs\ refine-module\ export-frontend\ scaffold-infra</code>	<code>aidd-master</code>
<code>enterprise inject <tipo> <nome> [--dir]</code>	<code>aidd-enterprise</code>
<code>enterprise verificar-drift [--dir]</code>	<code>aidd-enterprise</code>
<code>ops plan "<requisito>" [--pasta]</code>	<code>aidd-ops</code>
<code>ops plan --ferramentas-json <arquivo></code>	<code>aidd-ops</code>
<code>ops bootstrap --host <ip></code>	<code>aidd-ops</code>
<code>ops deploy</code>	<code>aidd-ops</code>
<code>ops preflight</code>	<code>aidd-ops</code>

A.3 Evolução e governança

Comando	O que faz
<code>melhoria init --pedido "<texto>" --nome "<n>"</code>	Etapa 1: análise com nota e evidência em docs/melhorias/
<code>plan init <nome></code>	Etapa 2: estrutura o plano em docs/planos/<nome>/
<code>orchestrate <plano> [--dry-run]</code>	Etapa 3: roteia ambiente, monta Plano de Voo e executa
<code>audit</code>	Delega a <code>pre-commit run --all-files</code>
<code>status</code>	Painel de ferramentas, habilidades e slash commands
<code>status --testes [--write]</code>	Telemetria de testes
<code>harness</code>	Diagnóstico de compatibilidade multi-harness
<code>preflight-host</code>	Diagnóstico do host

A.4 Gestão de componentes e dependências

Comando	O que faz
<code>components sync --tipo <tipo\ todos> [--ferramenta] [--dry-run] [--force]</code>	Distribui componentes aos assistentes
<code>components verify --tipo <tipo\ todos> [--ferramenta]</code>	Audita a distribuição sem escrever
<code>dependencia bootstrap [--tipo skills\ mcps\ todos] [--dry-run]</code>	Instala o que falta
<code>dependencia add-skill --nome --pacote --instalar --verificar</code>	Registra habilidade de terceiro
<code>dependencia add-mcp --nome --pacote --tipo stdio\ remote [...]</code>	Registra servidor MCP
<code>dependencia list</code>	Lista o registro
<code>dependencia verify</code>	Verifica a instalação (roda no início da sessão)

A.5 Flags globais

`--auto-bootstrap` instala `requirements.txt` automaticamente quando falta dependência, em vez de apenas exibir o banner educativo e sair com 1.

Apêndice B — Os contratos formais de handoff

Os cinco esquemas em [componentes/compartilhado/specs/](#) são a espinha dorsal da integração entre ferramentas. Todos seguem JSON Schema e são validados por [jsonschema](#) no orquestrador síncrono.

B.1 [handoff-planner-to-engine.schema.json](#)

Campos obrigatórios: [versao_schema](#), [fluxo_alvo](#) (inteiro: 1 = do zero puro, 2 = open-source, 3 = ponte low-code), [metadados_projeto](#), [quarteto_sine_qua_non](#), [arquitetura_alvo](#), [modulos_funcionais](#).

O [arquitetura_alvo](#) carrega o Padrão-Ouro: [padrao_frontend](#): [nextjs_typescript_tailwind](#), [padrao_backend](#): [fastapi_modular_vsa](#), e [persistencia](#) que varia por fluxo ([sqlite_wal](#) nos Fluxos 01 e 02, [postgresql](#) no Fluxo 03).

B.2 [handoff-engine-to-master.schema.json](#)

Campos obrigatórios: [versao_schema](#), [origem_engine](#) (qual motor produziu), [projeto_slug](#), [slices_geradas](#) (com [slice_nome](#), [caminho_src](#), [endpoints](#), [tabelas_sql](#)), [artefatos_frontend](#) (com [tecnologia](#), [paginas_geradas](#), [origem_design](#)), [testes_executados](#) (com [total](#), [passaram](#), [falharam](#), [zero_stubs](#)).

O campo [origem_design](#) registra a procedência visual: [custom_tdd](#) no Fluxo 01, [tailwind_standard](#) no Fluxo 02, [lovable_preserved](#) no Fluxo 03.

B.3 [handoff-master-to-enterprise.schema.json](#)

Campos obrigatórios: [versao_schema](#), [diretorio_projeto](#), [servidor_fastapi_ok](#), [quarteto_sine_qua_non_rotas](#), [componentes_para_blindagem](#).

B.4 [handoff-enterprise-to-ops.schema.json](#)

Campos obrigatórios: [versao_schema](#), [diretorio_projeto](#), [sha256_audit_ok](#), [drift_verificado](#), [manifesto_deploy](#) (com [tipo_runtime](#), [dockerfile_presente](#), [compose_presente](#), [portas_expostas](#)).

B.5 [plano-infraestrutura.schema.json](#)

O maior dos cinco (10,8 KB). Define o envelope de três fases — [fase_1_intake](#), [fase_2_curadoria](#), [fase_3_sizing](#) — compartilhado por [aidd-ops](#), [aidd-factory](#) e [aidd-planner](#). É o contrato que permite ao planner exportar diretamente para a fábrica sem tradução intermediária.

Apêndice C — Glossário

AIDD — AI-Driven Development. O nome do ecossistema e da família de práticas.

Agnosticidade (Supremacia Agnóstica) — Lei #6. Ausência de dependência de fornecedor em quatro dimensões: sistema operacional, assistente, modelo e protocolo.

Anti-NIH — Política de preferir integrar software consolidado a reimplementá-lo. NIH é not invented here.

Caveman (Protocolo Caveman Ultra) — Motor de compressão tri-fase que reduz a entrada em inglês telegráfico, raciocina em inglês comprimido e entrega em PT-BR de padrão alto.

Context-Purge — Descarte imediato do contexto de um subagente após a validação da sua tarefa.

Contexto delimitado (bounded context) — Fronteira de um módulo de negócio. Import direto entre contextos é proibido.

Determinismo — Regime de execução em que o resultado é computado por algoritmo, sem chamada de modelo. É o regime padrão do ecossistema (Lei #1).

Drift — Divergência entre o estado atual de um componente e o seu estado registrado/assinado. Detectado por [verificar-drift](#) e por [G_DRIFT_NUCLEO_COMPARTILHADO](#).

Fatia vertical (VSA, vertical slice architecture) — Módulo autônomo contendo todas as camadas de uma funcionalidade, do roteamento à persistência.

Handoff — Passagem de bastão entre duas etapas da esteira, sempre mediada por um documento JSON validado contra esquema.

Harness — O assistente de IA que hospeda a sessão (Claude Code, OpenCode, Cursor, Antigravity, entre outros).

Honestidade de Rótulo — Lei #8. Proibição de alegar certificação, cobertura ou medição além do resultado real.

Micro-ambiente — O [AGENTS.md](#) isolado de uma fase, contendo apenas as regras daquela fase.

Mônada Result — Tipo de retorno que encapsula sucesso ([Result.ok](#)) ou falha ([Result.fail](#)), impedindo que exceção crua vaze para a apresentação.

Portão (quality gate) — Script determinístico que retorna 0 (aprova) ou 1 (bloqueia). Lei #2.

PRÉ-PLANO — O intake interativo conduzido pelo [aidd-planner](#) antes de gerar o [PLANNER.json](#).

Protocolo delegado — Mecanismo pelo qual o pipeline usa o modelo do próprio assistente da sessão, sem abrir conexão própria com provedor.

Quarteto Sine Qua Non — Lei #10. Os quatro estúdios obrigatórios em todo projeto: [/docs](#), [/webhooks](#), [/mcp](#) e [/docs/guia](#).

Tríade Canônica — Os três fluxos de criação: [aidd-pure](#), [aidd-open](#) e [aidd-bridge](#).

Tokenomics — A disciplina de orçar, medir e auditar o consumo de tokens por fase.

Zero Stubs — Lei #5. Proibição de função vazia, [TODO](#), marcador de espaço ou retorno simulado em código de produção.

WAL (write-ahead logging) — Modo de operação do SQLite exigido pelo ecossistema para permitir leitura concorrente durante escrita.

Apêndice D — Mapa de arquivos-chave

Para quem quiser verificar as afirmações deste livro diretamente no repositório, esta é a rota mais curta para cada assunto.

Assunto	Arquivo
As treze leis invioláveis	AGENTS.md §2
A Tríade Canônica	AGENTS.md §3
Catálogo detalhado de portões e comandos	docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md
Padrão-Ouro de stack	docs/protocolos/PADRAO-OURO-STACK-TECNOLOGICA.md
Ciclo de teste de ferramenta	docs/protocolos/PROTOCOLO-TESTES-FERRAMENTAS.md
Agnosticidade de componentes	docs/protocolos/05-09-2026_protocolo-agnosticidade-componentes.md
Validação de testes reais	docs/protocolos/06-09-2026_protocolo-validacao-testes-reais.md
A CLI unificada	ecossistema.py
O orquestrador da tríade	scripts/orquestrador_sincrono.py
Contratos de handoff	componentes/compartilhado/specs/
Protocolo Caveman	componentes/compartilhado/src-core/caveman_protocol.py
Fatiamento de contexto	core/context_slicer.py
Roteador MCP dinâmico	core/mcp_dynamic_router.py
Livro-razão cognitivo	core/cognitive_ledger.py
Orçamento de tokens por fase	tools/aidd-generator/config/token_budgets.json
Auditoria de tokenomics	tools/aidd-generator/scripts/gates/G_TOKENOMICS.py
Micro-ambientes das 8 fases	tools/aidd-generator/scripts/phases/phase_*/AGENTS.md
Configuração dos portões no commit	.pre-commit-config.yaml
Hooks de assistente	.claude/settings.json
Hooks canônicos compartilhados	componentes/compartilhado/hooks/
Manifesto multi-harness	gates/manifesto_harnesses.json
Baseline do núcleo compartilhado	gates/baseline_nucleo_compartilhado.json
Memória viva do projeto	MEMORY.md
Índice de planos	docs/planos/INDEX.md

Apêndice E — Estado honesto na data de geração

A Lei #8 exige que este livro registre não apenas o que funciona, mas o que está pendente. Esta é a lista consolidada de tudo que o texto sinalizou como incompleto, vermelho ou aguardando decisão.

Item	Estado	Onde está registrado
G_QUARTETO_SINE_QUA_NON auto-descoberta	Cobre 2 de 3 fluxos canônicos com exemplo real (falta saída real do Fluxo 02/03)	gates/G_QUARTETO_SINE_QUA_NON.py , saída do próprio portão
Campos de telemetria do orquestrador síncrono	Parte dos payloads de handoff é montada com valores fixos, não medidos	scripts/orquestrador_sincrono.py , etapas 3, 5 e 7
Discovery Engine completo do aidd-factory	Só o subconjunto determinístico (nicho dinâmico) está implementado	docs/features/v2_arquitetura-aidd-ops-factory.md §7.1 e §9.1
.gemini/skills/ como mecanismo	Sincronizado, mas confirmado: false — o mecanismo real são as extensões	docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md §5
Freebuff	Instalado, sem modo não interativo para validação automatizada	docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md §5

POR QUE ESTE APÊNDICE EXISTE

Um documento corporativo que só mostra o que funciona é um documento de vendas. A Lei 8 do ecossistema — Honestidade de Rótulo — proíbe alegar certificação ou cobertura além do resultado real, e essa proibição não faz exceção para a documentação sobre o próprio ecossistema.

Cada linha da tabela acima tem um arquivo do repositório que a sustenta, e cada uma delas foi encontrada lendo o código e a configuração, não interpretando intenções. É assim que se verifica se um livro sobre um sistema descreve o sistema ou descreve o que alguém gostaria que ele fosse.